Solver specification:

`highOrderElectroActivationFoamImplicitPETScDistributed`

A high-order, implicit, Jacobian-free Newton–Krylov monodomain solver in OpenFOAM with a switchable Eigen/PETSc linear-algebra back-end and distributed-memory linear algebra

Pablo

October 18, 2026

Abstract

This report specifies the algorithms implemented in the OpenFOAM solver:

`highOrderElectroActivationFoamImplicitPETSc`.

The solver advances the monodomain equations of cardiac electrophysiology coupled to the ten Tusscher–Noble–Noble–Panfilov (TNNP, 2004) cellular ionic model. The solver has several features:

- The diffusion operator is discretised with the OpenFOAM cell-centred finite-volume framework, optionally enhanced to a high-order Local Regression Estimators (LRE) reconstruction;
- The ionic source is evaluated either at cell centres or at the cell quadrature points (required for high order);
- There are different ways to integrate the reaction ODE equations:
  (i) at each quadrature point used to evaluate the ionic source;
  (ii) only at the centres, reconstructing them afterwards wherever the ionic source has to be evaluated;
  (iii) a front-aware hybrid that does (i) only on the propagating front and (ii) everywhere else (Section 5);
- Time integration follows a θ scheme (backward Euler or Crank–Nicolson) or BDF1–4 with an ESDIRK3 startup (since 2026-10, Section 13);
- the per-cell ionic ODE uses the solver's own integrator (adaptive `RKF45` by default, `RK4` or `Euler`) with configurable tolerances, driven by a polynomial in time of V_{m} across the step;
- The mass operator can be `lumped` or high-order `consistent`;
- the resulting coupled nonlinear system can be solved by Picard fixed-point iteration, a diagonal linearisation of I_{ion}, or a Jacobian-Free Newton–Krylov (JFNK) method with Armijo backtracking line search, an initial guess by linear extrapolation and a physical bracket *clamp* on the input to the ionic ODE;
- the linear system of the assembled branches (Picard / diagonalIion) and the JFNK inner system are solved with a *switchable* back-end via the `linearSolverBackend` key: either Eigen's sparse factorisations (`SparseLU`/`BiCGSTAB`) and an in-house restarted GMRES with modified Gram–Schmidt, or PETSc's KSP/PC solvers (including a GMRES over a matrix-free `MatShell` for JFNK). Matrix assembly and storage always use `Eigen::SparseMatrix`;
- activation times are recorded by linear interpolation across the first threshold crossing and the benchmark points are sampled by inverse-distance weighting.

All algorithmic parameters are exposed through the `constant/spatialIntegrationProperties` and `constant/timeIntegrationProperties` dictionaries, and the Python driver `run_convergence.py` generates these dictionaries from a single configuration file. This solver integrates both linear-algebra back-ends (Eigen and PETSc) in a single executable, selectable at run time with `linearSolverBackend` / `jfnkLinearSolverBackend` (Section 7). There is also a simpler sibling solver, `highOrderElectroActivationFoamImplicit`, with an Eigen-only back-end and a semi-implicit single-Picard-iteration scheme (Section E for the performance differences).

This `...Distributed` variant additionally makes the linear algebra *genuinely distributed*: the mesh is decomposed with `decomposePar`, the operators are assembled with local rows and global columns and become PETSc `MPIAIJ` matrices on `PETSC_COMM_WORLD`, and every matrix–vector product goes through `MatMult` instead of Eigen. The algorithms of Sections 5–7 are unchanged; Section D describes what the decomposition requires, how a parallel operator is verified, and the inconsistency that was measured in the stencil selection of the high-order library (Section D.5).

Status as of 2026-10-18. Since October the code this solver shares with the MMS solver lives in `src/monodomainCommon/`, and the solver gained BDF schemes, the history-based V_{m} driver, a common nonlinear stopping test, right preconditioning of the KSP, the V_{m} history at the Gauss points and the fix of the hybrid's temporal floor. Section 13 collects those changes and the new results (E02, E35, E36 and M02); the earlier sections have been corrected where they had become false. The result tables of Section 14 **predate** those changes.

Contents

1 Notation **4**

2 Governing equations **4**
- 2.1 Monodomain PDE . . . 4
- 2.2 Ionic ODE . . . 4
- 2.3 Manufactured MMS equation used for convergence transfer . . . 5
- 2.4 External stimulus . . . 5

3 Time discretisation: the θ-scheme **5**

4 Spatial discretisation **6**
- 4.1 Standard orthogonal FV Laplacian . . . 6
- 4.2 High-order LRE diffusion . . . 6
- 4.3 Mass matrices . . . 6
- 4.4 High-order reconstruction at ionic quadrature points . . . 7
- 4.5 High-order cell averaging of I_{ion} . . . 7

5 State integration at the ionic quadrature points **7**
- 5.1 Two modes . . . 7
- 5.2 Front-aware hybrid . . . 7
- 5.3 State reconstruction limiter . . . 8
- 5.4 Dictionary keys . . . 8

6 Nonlinear solver **9**
- 6.1 Shared residuals and stopping tests . . . 9
- 6.2 Picard fixed-point method . . . 9
- 6.3 Diagonal I_{ion} linearisation . . . 10
- 6.4 Jacobian-Free Newton–Krylov . . . 10
- 6.5 JFNK inner Krylov . . . 10
- 6.6 Armijo backtracking line search . . . 11
- 6.7 Initial guess, ODE clamp and rollback . . . 11

7 Linear solver: Eigen and PETSc back-ends **11**

8 Ionic ODE integration **12**

9 TNNP numerical protection **12**

10 Activation tracking and benchmarks **13**

11 Solver flow chart **14**

12 Manufactured driver: `run_convergence.py` 16
12.1 Geometry and parameter sweep . . . 16
12.2 Nonlinear method and convergence controls . . . 17
12.3 JFNK and diagonal-Iion controls . . . 18
12.4 Linear PDE solver . . . 18
12.5 State ODE integrator . . . 19
12.6 LRE high-order parameters per p-level . . . 19
12.7 Diagnostics and run-control . . . 19

13 October 2026 changes and new results 20
13.1 Code shared with the MMS solver . . . 20
13.2 BDF1–4 with an ESDIRK3 startup . . . 20
13.3 The V_{m} driver of the ODEs . . . 20
13.4 The V_{m} history at the Gauss points . . . 21
13.5 The hybrid's temporal floor, and its fix . . . 21
13.6 Other changes . . . 22
13.7 Defaults: solver and harness . . . 22
13.8 New results . . . 22

14 Results 23
14.1 2D Niederer-style benchmark . . . 23
14.1.1 Computational cost quantities . . . 24
14.1.2 Structured hexahedral mesh, no hybrid (`frontAwareHybrid false`) . . . 24
14.1.3 Structured hexahedral mesh, front-aware hybrid (`frontAwareHybrid true`) . . . 26
14.1.4 Unstructured triangular mesh, no hybrid . . . 27
14.1.5 Unstructured triangular mesh, front-aware hybrid . . . 28
14.2 3D Niederer-style benchmark . . . 32

A Dimension table 33

B File map 33

C Linear-algebra back-ends and sibling solver 34

D Distributed-memory parallelisation 34
D.1 Matrix layout: local rows, global columns . . . 35
D.2 What has to cross a processor boundary . . . 35
D.3 Global reductions . . . 36
D.4 Verifying a parallel operator . . . 36
D.5 Partition dependence of the MLS stencils . . . 37
D.6 Verification cases: what was run and where it lives . . . 38

E Performance optimisations 40
E.1 Matrix and factorisation reuse (Phase B) . . . 40
E.2 Diffusion preconditioner for JFNK (A3) . . . 40
E.3 Parallel ionic ODE integration (C1) . . . 40
E.4 AMG preconditioners (C3) . . . 41

F Performance experiments 41
F.1 Phase B: identical and faster in Picard . . . 41
F.2 A3: diffusion preconditioner in JFNK . . . 42
F.3 C1: ODE parallelisation (does not help in this problem) . . . 42
F.4 Internal PETSc/BLAS threading (counterproductive in Picard) . . . 42
F.5 Linear solver: AMG vs. ILU (the solve is not the bottleneck) . . . 42

F.6 Synthesis and recommendations 43

1 Notation

Symbol	Meaning	Units
$\Omega \subset \mathbb{R}^d$	Spatial domain ($d \in \{2, 3\}$)	m
$\partial\Omega$	Insulated boundary (zero-flux Neumann)	—
$t \in (0, T]$	Time	s
$V_{\mathrm{m}}(\mathbf{x}, t)$	Transmembrane voltage	V
$\mathbf{s}(\mathbf{x}, t) \in \mathbb{R}^{17}$	TNNP ionic states	mixed
$I_{\mathrm{ion}}(V_{\mathrm{m}}, \mathbf{s})$	Total ionic current density	$\mathrm{V\,s^{-1}}$
$I_{\mathrm{ext}}(\mathbf{x}, t)$	External stimulus current density	$\mathrm{A\,m^{-3}}$
χ	Membrane surface-to-volume ratio	$\mathrm{m^{-1}}$
C_m	Membrane capacitance per unit area	$\mathrm{F\,m^{-2}}$
$\mathbf{D}(\mathbf{x})$	Bulk conductivity tensor	$\mathrm{S\,m^{-1}}$
$\theta \in \{0.5, 1\}$	Time-stepping parameter (CN or BE)	—
Δt	Outer time-step length	s
N_c	Number of mesh cells	—
$\mathsf{V}^n \in \mathbb{R}^{N_c}$	Vector of cell-averaged V_{m} at time t^n	V
$\mathbf{M}, \mathbf{K}, \mathbf{L}$	Mass, stiffness, scaled diffusion matrix	—
$\mathbf{A}, \mathbf{B}$	Implicit LHS / RHS operators of the θ-scheme	—
$\mathsf{F}(\mathsf{x})$	Discrete nonlinear residual to be zeroed	—

2 Governing equations

2.1 Monodomain PDE

The bulk voltage is propagated by the standard monodomain reaction–diffusion equation [1, 2]. In physical current-density notation one may write

$$\chi\, C_m \frac{\partial V_{\mathrm{m}}}{\partial t} \;=\; \nabla\cdot\big(\mathbf{D}\nabla V_{\mathrm{m}}\big) - \chi C_m\, I_{\mathrm{ion}}(V_{\mathrm{m}}, \mathbf{s}) + I_{\mathrm{ext}} \quad \text{in } \Omega \times (0, T], \tag{1}$$

subject to insulated boundaries $\mathbf{n} \cdot (\mathbf{D}\nabla V_{\mathrm{m}}) = 0$ on $\partial\Omega \times (0, T]$, and initial condition $V_{\mathrm{m}}(\mathbf{x}, 0) = -84\,\mathrm{mV}$. The conductivity tensor $\mathbf{D}$ is symmetric positive-definite and either uniform or read as a `volTensorField` from disk. The factor χC_m multiplying I_{ion} in (1) reflects the convention used by the ionic-model interface: the solver stores `Iion` as a voltage rate with dimensions $\mathrm{V\,s^{-1}}$, not as an $\mathrm{A\,m^{-3}}$ volumetric current.

Dividing (1) by χC_m gives the form actually used inside the solver,

$$\frac{\partial V_{\mathrm{m}}}{\partial t} \;=\; \mathcal{L} V_{\mathrm{m}} \;+\; s(V_{\mathrm{m}}, \mathbf{s}), \qquad \mathcal{L} \;=\; \frac{1}{\chi C_m}\nabla\cdot(\mathbf{D}\nabla\cdot), \quad s \;=\; -I_{\mathrm{ion}} + \frac{I_{\mathrm{ext}}}{\chi C_m}. \tag{2}$$

This is exactly what `rhsFromIion` assembles in the solver.

2.2 Ionic ODE

At every spatial sampling point the TNNP state vector $\mathbf{s} \in \mathbb{R}^{17}$ evolves according to

$$\frac{\mathrm{d}\mathbf{s}}{\mathrm{d}t} \;=\; \mathbf{f}_{\mathrm{TNNP}}\big(V_{\mathrm{m}}, \mathbf{s}; \texttt{tissue}\big), \qquad \mathbf{s}(0) = \mathbf{s}_0, \tag{3}$$

where `tissue` selects `epicardialCells`, `mCells` or `endocardialCells`. The 17 tracked states are

$$\mathbf{s} = (V,\ K_i,\ Na_i,\ Ca_i,\ Xr_1,\ Xr_2,\ Xs,\ m,\ h,\ j,\ d,\ f,\ fCa,\ s,\ r,\ g,\ Ca_{SR}).$$

The total ionic current returned to the PDE is the algebraic `Iion_cm` of TNNP [1].

2.3 Manufactured MMS equation used for convergence transfer

The convergence driver in `tutorials_electro/highOrderManufacturedFDAImplicitJfNK` solves a manufactured problem with the same numerical structure as the electro-activation solver: implicit monodomain-like PDE, local state ODEs, nonlinear ionic source, optional LRE reconstruction for V_{m} and I_{ion}, and selectable `Picard`/`diagonalIion`/`JFNK` nonlinear coupling. The analytical fields are

$$V_{\mathrm{m}}^{\star}(\mathbf{x},t) = \sqrt{1+t}\,F(\mathbf{x}), \tag{4}$$
$$u_1^{\star}(\mathbf{x},t) = (1+t)\,G(\mathbf{x}) + V_{\mathrm{m}}^{\star}(\mathbf{x},t), \tag{5}$$
$$u_2^{\star}(\mathbf{x},t) = \frac{1}{(1+t)\sqrt{G(\mathbf{x})}}, \tag{6}$$
$$u_3^{\star}(\mathbf{x},t) = 0, \tag{7}$$

with, for example, $F = \cos(\pi x)\cos(2\pi y)$ and $G = 1+x\,y^2$ in 2D. In 3D, $F = \cos(\pi x)\cos(2\pi y)\cos(3\pi z)$ and $G = 1 + x\,y^2 z^3$. The state equations are

$$\dot{u}_1 = (u_1+u_3-V_{\mathrm{m}})^2 u_2^2 + \frac{1}{2}(u_1+u_3-V_{\mathrm{m}})u_2^2(V_{\mathrm{m}}-u_3), \tag{8}$$
$$\dot{u}_2 = -(u_1+u_3-V_{\mathrm{m}})u_2^3, \tag{9}$$
$$\dot{u}_3 = 0. \tag{10}$$

The manufactured ionic source is

$$I_{\mathrm{ion}}^{\mathrm{MMS}}(V_{\mathrm{m}},\mathbf{u}) = -\frac{1}{2}(u_1+u_3-V_{\mathrm{m}})u_2^2(V_{\mathrm{m}}-u_3) + \frac{\beta}{\chi C_m}(V_{\mathrm{m}}-u_3), \tag{11}$$

and the PDE source used by the manufactured solver is $-I_{\mathrm{ion}}^{\mathrm{MMS}}$. The coefficient β is the spatial eigenvalue of the trigonometric manufactured field under the constant conductivity tensor:

$$\beta = \begin{cases} -\pi^2 D_{xx}, & d=1, \\ -\pi^2(D_{xx}+4D_{yy}), & d=2, \\ -\pi^2(D_{xx}+4D_{yy}+9D_{zz}), & d=3. \end{cases}$$

Thus the MMS case is not intended to reproduce TNNP physiology; it is a controlled problem with a known exact solution that exercises the same discrete operators and nonlinear solution paths used by this solver.

2.4 External stimulus

The external current is a time- and region-windowed step:

$$I_{\mathrm{ext}}(\mathbf{x},t) \;=\; \begin{cases} I_0 & \mathbf{x}\in\Omega_{\mathrm{stim}}^{(b)},\;\; t\in[t_0^{(b)},\,t_0^{(b)}+\tau],\;\; b=1,\dots,B, \\ 0 & \text{otherwise}, \end{cases} \tag{12}$$

with $I_0 = 50\,000\,\mathrm{A\,m^{-3}}$ and $\tau = 2\,\mathrm{ms}$ as Niederer-benchmark defaults.

3 Time discretisation: the θ-scheme

Time is advanced from t^n to $t^{n+1} = t^n + \Delta t$ by

$$\frac{\mathsf{V}^{n+1}-\mathsf{V}^n}{\Delta t} \;=\; \theta\,\mathbf{L}\,\mathsf{V}^{n+1} + (1-\theta)\,\mathbf{L}\,\mathsf{V}^n + \theta\,\mathbf{s}^{n+1} + (1-\theta)\,\mathbf{s}^n. \tag{13}$$

`implicitScheme`	θ	Scheme
`backwardEuler`	1	First-order, L-stable
`crankNicolson`	1/2	Second-order, A-stable

After multiplication by Δt^{-1} the implementation forms

$$\left[\mathbf{M}/\Delta t \;-\; \theta\,\mathbf{L}\right]\mathsf{V}^{n+1} \;=\; \left[\mathbf{M}/\Delta t \;+\; (1-\theta)\,\mathbf{L}\right]\mathsf{V}^{n} \;+\; \theta\,\mathbf{s}^{n+1} \;+\; (1-\theta)\,\mathbf{s}^{n}, \tag{14}$$

with the two Eigen `SparseMatrix<scalar,RowMajor>` operators

$$\mathbf{A} \;=\; \mathbf{M}/\Delta t \;-\; \theta\,\mathbf{L}, \qquad \mathbf{B} \;=\; \mathbf{M}/\Delta t \;+\; (1-\theta)\,\mathbf{L}. \tag{15}$$

$\mathbf{A}$ and $\mathbf{B}$ are rebuilt at every time step by copying $\mathbf{M}$, scaling by Δt^{-1} and adding $\pm\theta\mathbf{L}$. When $\theta = 1$ (BE) the explicit half of the diffusion vanishes and $\mathbf{B} = \mathbf{M}/\Delta t$. The scaled diffusion is $\mathbf{L} = \mathbf{K}/(\chi C_m)$.

BDF1–4 (2026-10). `implicitScheme` also accepts `BDF1` (bit-identical to `backwardEuler`), `BDF2`, `BDF3` and `BDF4`. They end up in the same pair of operators, with $a_0/\Delta t$ instead of $1/\Delta t$ in front of $\mathbf{M}$ and the history added as a single field; the steps a BDF-k cannot take by itself are taken with ESDIRK3. See Section 13.2. Contrary to the above, $\mathbf{A}$ is no longer rebuilt every step: it is reused while Δt and the scheme do not change (Section E.1).

4 Spatial discretisation

4.1 Standard orthogonal FV Laplacian

`assembleStandardOrthogonalStiffnessMatrix` computes the classical 5/7-point Laplacian. Per internal face f with face area $\mathbf{S}_f$,

$$a_f \;=\; \frac{|\mathbf{S}_f|\,(\mathbf{n}_f \cdot \mathbf{D}_f \cdot \hat{\mathbf{d}}_f)}{|\mathbf{d}_f|}, \quad \mathbf{D}_f = \tfrac{1}{2}\,(\mathbf{D}_{\text{own}} + \mathbf{D}_{\text{nei}}), \tag{16}$$

contributing $-a_f/V_{\text{own}}$ to $\mathbf{K}[\text{own},\text{own}]$, $+a_f/V_{\text{own}}$ to $\mathbf{K}[\text{own},\text{nei}]$, and the transposed pair to the neighbour row. `zeroGradient`/`empty` patches contribute zero flux. `fixedValue` patches contribute to the owner diagonal only.

4.2 High-order LRE diffusion

When `useHighOrder_Vm = true` and $d > 1$, `assembleHighOrderStiffnessMatrix` integrates the flux on a face quadrature with reconstructed gradients:

$$\Phi_f \;=\; \sum_{q\in\mathcal{Q}_f} w_q\,|\mathbf{S}_f|\,\mathbf{n}_f \cdot \mathbf{D}_f \cdot \Big(\sum_{j\in\mathcal{S}_f} \mathbf{g}_{f,q,j}\,\mathsf{V}_j\Big), \tag{17}$$

where $\mathcal{S}_f$ is the LRE face stencil ($\sim$ 15–50 cells) and $\mathbf{g}_{f,q,j}$ = `QRGradFaceGPCoeffs`. The contribution of each V_j is added to row `own` with weight $+\Phi_{f,j}/V_{\text{own}}$ and to row `nei` with weight $-\Phi_{f,j}/V_{\text{nei}}$, yielding a negative semi-definite Laplacian.

4.3 Mass matrices

Two options:

Lumped (`massMatrix lumped`). `assembleDiagonalMassMatrix` writes $\mathbf{M} = I$.

Consistent HO (`massMatrix consistent`). `assembleConsistentMassMatrixHO` fills

$$\mathbf{M}[i,j] \;=\; \sum_{q\in\mathcal{Q}_i} \frac{w_q}{\sum_{q'} w_{q'}}\,\phi_j(\mathbf{x}_q), \quad \phi_j(\mathbf{x}_q) = \delta_{ij} + \mathbf{g}_{i,j}\cdot\mathbf{d}_q + \tfrac{1}{2}\,\mathbf{H}_{i,j} : (\mathbf{d}_q \otimes \mathbf{d}_q) + \tfrac{1}{6}\,\mathbf{T}^{(3)}_{i,j} \vdots (\mathbf{d}_q \otimes \mathbf{d}_q \otimes \mathbf{d}_q), \tag{18}$$

with $\mathbf{d}_q = \mathbf{x}_q - \mathbf{C}_i$. A safety guard silently downgrades to the lumped form when `useHighOrder_Vm = false` or $d \le 1$.

4.4 High-order reconstruction at ionic quadrature points

If `useHighOrder_Iion = true`, the ionic ODE is integrated at cell quadrature points. Vm is reconstructed there by the same Taylor expansion as in (18):

$$V_{\mathrm{m}}(\mathbf{x}_q) \;\approx\; V_{\mathrm{m},c} + (\nabla V_{\mathrm{m}})_c \cdot \mathbf{d}_q + \tfrac{1}{2}\,\mathbf{H}_c : (\mathbf{d}_q \otimes \mathbf{d}_q) + \tfrac{1}{6}\,\mathbf{T}_c^{(3)} \vdots (\mathbf{d}_q \otimes \mathbf{d}_q \otimes \mathbf{d}_q), \tag{19}$$

implemented in `reconstructVmAtIionIntegrationPoints`.

4.5 High-order cell averaging of I_{ion}

`averageIionIntegrationPointsToCells` contracts the per-QP I_{ion} back to cells:

$$I_{\mathrm{ion},i} \;=\; \frac{\sum_{q\in\mathcal{Q}_i} w_q\, I_{\mathrm{ion},q}}{\sum_{q\in\mathcal{Q}_i} w_q}. \tag{20}$$

5 State integration at the ionic quadrature points

The high-order quadrature (19) evaluates $I_{\mathrm{ion}}(V_{\mathrm{m}}, \mathbf{s})$ at the cell quadrature points $\mathbf{x}_q$, so it needs the ionic state vector $\mathbf{s}$ there as well. *How* the states reach the quadrature points is governed by `stateIntegrationMode` and, orthogonally, by `frontAwareHybrid`. This choice turns out to be decisive for a propagating cardiac front, and this section documents both the algorithm and the reason it matters.

5.1 Two modes

Let $\mathcal{Q}_i = \{\mathbf{x}_q\}$ be the quadrature set of cell Ω_i with weights w_q, and let $\mathbf{s}_c$ denote a cell-centred state.

`gaussPointODE` (legacy). The stiff ODE (3) is integrated *independently at every quadrature point*, driven by the locally reconstructed potential:

$$\mathbf{s}_q^{n+1} = \mathrm{ODEsolve}\big(\mathbf{s}_q^{n},\, V_{\mathrm{m}}(\mathbf{x}_q); \Delta t\big), \qquad I_{\mathrm{ion},q} = \phi\big(V_{\mathrm{m}}(\mathbf{x}_q), \mathbf{s}_q^{n+1}\big). \tag{21}$$

Cost: N_q stiff integrations per cell.

`cellCentredReconstruct`. The ODE is integrated *once per cell* at the cell centre, and the resulting cell-centred states are reconstructed to the quadrature points by a dedicated LRE (`LREInterp_states`, order set by `highOrderCoeffs/LRECoeffs_states`):

$$\mathbf{s}_c^{n+1} = \mathrm{ODEsolve}\big(\mathbf{s}_c^{n},\, \overline{V}_{m,c}; \Delta t\big), \quad \widehat{\mathbf{s}}(\mathbf{x}_q) = \mathcal{R}^{(p)}\big[\{\mathbf{s}_{c'}^{n+1}\}\big](\mathbf{x}_q), \quad I_{\mathrm{ion},q} = \phi\big(V_{\mathrm{m}}(\mathbf{x}_q), \widehat{\mathbf{s}}(\mathbf{x}_q)\big). \tag{22}$$

Cost: one stiff integration per cell – a factor N_q cheaper, which for the expensive TNNP integrations is the motivation. The two modes differ only in the argument of the non-linear map $V_{\mathrm{m}} \mapsto \mathbf{s}$: pointwise $V_{\mathrm{m}}(\mathbf{x}_q)$ in (21), versus the cell average $\overline{V}_{m,c}$ in (22).

5.2 Front-aware hybrid

`frontAwareHybrid` (default `false`, orthogonal to `stateIntegrationMode`) combines the two modes: it solves the ODE at the Gauss points in the thin band of *front* cells and at the cell centre (with reconstruction) everywhere else. A cell i is a *front* cell when its sub-cell V_{m} range exceeds `hybridFrontVmRange` or it is stimulated,

$$\mathrm{front}_i \iff \Big(\max_q V_{\mathrm{m}}(\mathbf{x}_q) - \min_q V_{\mathrm{m}}(\mathbf{x}_q)\Big) > \texttt{hybridFrontVmRange} \;\vee\; (I_{\mathrm{ext}})_i \neq 0. \tag{23}$$

The front states are carried at the Gauss points (as in `gaussPointODE`); **since 2026-10-16 the smooth cells carry their own cell state**, `hybridCellStates`, as pure CC does. Before, that cell state was rebuilt every step as the quadrature average of the Gauss states, which put a floor on the temporal convergence (Section 13.5). Per nonlinear evaluation the branch is:

(a) front cells: one ODE per Gauss point, local V_{m} (as (21));

(b) smooth cells: one ODE at the cell centre from its cell state at the start of the step (the Gauss average is used only once, when a cell leaves the front);

(c) reconstruct the smooth-cell states to their Gauss points, front Gauss states left untouched;

(d) evaluate I_{ion} at all Gauss points and average to the cell.

Freezing per time step (avoiding a limit cycle). The front classification (23) is a *solution-dependent discrete switch*. If recomputed inside the nonlinear loop, cells near the threshold flip front↔smooth between iterations; because the two branches compute I_{ion} by different paths, the source term jumps discretely and the Picard fixed-point map becomes discontinuous, producing a non-convergent *period-2 limit cycle* (observed as $\mathrm{iter}_{k+2} \equiv \mathrm{iter}_k$ in `nonlinearResiduals.dat`). The classification is therefore computed *once per time step* from the committed V_{m}^n and *frozen* for the whole nonlinear solve, and the band is dilated by `hybridFrontDilation` cell layers (via `mesh.cellCells()`) so the front stays inside the Gauss-resolved zone as it advances. Freezing is safe because the front moves far less than one cell per step (for the 2D benchmark, $\sim 0.09\,\mathrm{mm/ms}$, i.e. ~ 0.009 cell per step at $\Delta t = 10^{-4}\,\mathrm{s}$, $\Delta x = 0.5\,\mathrm{mm}$).

5.3 State reconstruction limiter

High-order ($p3$) reconstruction of the stiff TNNP states across an under-resolved front overshoots (Gibbs), and a state driven far outside its physical range can overflow the ionic algebra (observed as a PETSc `SIGFPE`). `stateReconstructLimiter` (default `false` since 2026-10-06, in the solver and in the harness; when requested, active for `cellCentredReconstruct` and the hybrid smooth cells, independent of `frontAwareHybrid`) applies a Barth–Jespersen-style bound: each reconstructed value is clipped to the range of the cell and its face neighbours,

$$\widehat{s}(\mathbf{x}_q) \;\leftarrow\; \mathrm{clip}\Big(\widehat{s}(\mathbf{x}_q),\; \min_{c' \in \{i\} \cup \mathcal{N}(i)} s_{c'},\; \max_{c' \in \{i\} \cup \mathcal{N}(i)} s_{c'}\Big). \tag{24}$$

No new extremum is created, so the reconstructed states stay within the physically realised (ODE-evolved) range and the ionic algebra never sees a pathological input; in smooth regions the reconstruction already lies within the range and the limiter is inactive, preserving the order there.

Why it is off. At the front, clipping to the neighbour range drops the reconstruction to first order exactly where it matters, and no run of the current suite needs it to finish. It remains available if a case overflows; without it the hybrid can hit `SIGFPE` in extreme configurations (seen while testing a hysteresis in the front classification, which was discarded).

5.4 Dictionary keys

Key (in `implicitHOCoeffs`)	Meaning	Default
`stateIntegrationMode`	`gaussPointODE` / `cellCentredReconstruct`	`cellCentredReconstruct`
`frontAwareHybrid`	Gauss ODEs on the front, reconstruct elsewhere	`false`
`hybridFrontVmRange`	sub-cell V_{m} range flagging a front cell [V]	5×10^{-3}
`hybridFrontDilation`	cell layers added around the front band	2
`stateReconstructLimiter`	Barth–Jespersen limiter on the reconstruction	`false`
`stateODEDriver`	V_{m} polynomial driving the ODEs: `history` / `linear`	`history`
`stateODEGaussPointHistory`	the history on the Gauss-point paths too	`false`

`LRECoeffs_states` (in `highOrderCoeffs`) sets the reconstruction order; it is required by `cellCentredReconstruct` and by the hybrid.

6 Nonlinear solver

At each outer time step the unknown is the new voltage vector $\mathsf{x} = \mathsf{V}^{n+1}$. For any candidate x the solver first reconstructs the corresponding V_{m} field, then calls `evaluateNonlinearFields`. That routine resets the ionic states to their time-step-start values $\mathbf{s}^n$, integrates the local ODEs from t^n to t^{n+1} using the candidate voltage, obtains $I_{\mathrm{ion}}(\mathsf{x})$, and optionally averages quadrature-point values back to cells. This gives the nonlinear residual

$$\mathsf{F}(\mathsf{x}) \;=\; \mathbf{A}\,\mathsf{x} \;-\; \mathsf{r}\big(I_{\mathrm{ion}}(\mathsf{x})\big), \qquad \mathsf{x} = \mathsf{V}^{n+1}, \tag{25}$$

with

$$\mathsf{r}(I_{\mathrm{ion}}) = \mathbf{B}\,\mathsf{V}^n + \theta\left[-I_{\mathrm{ion}}(\mathsf{x}) + I_{\mathrm{ext}}^{n+1}/(\chi C_m)\right] + (1-\theta)\left[-I_{\mathrm{ion}}^n + I_{\mathrm{ext}}^n/(\chi C_m)\right]. \tag{26}$$

The three nonlinear methods differ only in how they drive $\mathsf{F}(\mathsf{x})$ toward zero.

6.1 Shared residuals and stopping tests

Every method records the same diagnostics:

$$R_{\mathrm{cpl}} = \frac{\|\mathsf{F}(\mathsf{x}_k)\|_2}{\|\mathsf{x}_k\|_2 + \epsilon}, \tag{27}$$

$$R_{V_{\mathrm{m}}} = \frac{\|\mathsf{V}_k - \mathsf{V}_{k-1}\|_2}{\|\mathsf{V}_{k-1}\|_2 + \epsilon}, \tag{28}$$

$$R_{I_{\mathrm{ion}}} = \frac{\|I_{\mathrm{ion},k} - I_{\mathrm{ion},k-1}\|_2}{\|I_{\mathrm{ion},k-1}\|_2 + \epsilon}, \tag{29}$$

$$R_{\mathbf{s}} = \max_j \frac{\|\mathbf{s}_{j,k} - \mathbf{s}_{j,k-1}\|_2}{\|\mathbf{s}_{j,k-1}\|_2 + \epsilon}. \tag{30}$$

The actual implementation uses `relativeL2Difference` for $R_{V_{\mathrm{m}}}$ and $R_{I_{\mathrm{ion}}}$, and `maxStateRelativeL2Difference` for the state maximum. A nonlinear iteration can stop only after `minNonlinearIterations`; convergence then requires `nonlinearTolerance`, `nonlinearVmTolerance` and `nonlinearIionTolerance`. The state criterion `nonlinearStatesTolerance` is included only when `nonlinearRequireStatesConvergence=true`.

One test, with a floor exit (2026-10). The three methods and the ESDIRK stages stop with the same shared function, `nonlinearIterationExit`: first the increments must pass; then, if $R_{\mathrm{cpl}} \le$ `nonlinearTolerance` the iteration has converged, and if R_{cpl} did not fall since the previous iteration it stops at the *floor*: the iterate is as converged as the operator allows (the true residual a preconditioned Krylov solve leaves, or round-off). Floor exits are counted and reported in the log. The increment tests are never loosened.

6.2 Picard fixed-point method

The Picard branch treats the source as lagged during the PDE solve. At iteration k the states and ionic source are computed from x_k, then the linear PDE

$$\mathbf{A}\,\mathsf{V}_* = \mathsf{r}\big(I_{\mathrm{ion}}(\mathsf{x}_k)\big) \tag{31}$$

is solved with the Eigen sparse solver selected by `linearSolver`. The next candidate is relaxed,

$$\mathsf{x}_{k+1} = \mathsf{x}_k + \omega(\mathsf{V}_* - \mathsf{x}_k), \qquad \omega = \texttt{nonlinearRelaxation}. \tag{32}$$

After the update the solver re-evaluates states and I_{ion} at x_{k+1} and checks the residuals above. In this electro-activation solver, unlike a pure MMS Picard study, a failed Picard solve rolls back unless `nonlinearAcceptUnconverged=true`.

Advantages. Picard is simple, cheap per nonlinear iteration, and uses the same linear PDE solver as the classical implicit formulation. It is a useful baseline and is often adequate when Δt is small or the ionic source is weakly nonlinear over one time step.

Disadvantages. The source derivative with respect to V_{m} is ignored in the PDE matrix, so convergence can be slow near the action-potential upstroke or for large Δt. A small residual may require many iterations, and under-relaxation may be needed for robustness.

6.3 Diagonal I_{ion} linearisation

The `diagonalIion` branch is Picard plus a local finite-difference approximation of the source derivative. Define

$$S(V_{\mathrm{m}}) = -I_{\mathrm{ion}}(V_{\mathrm{m}}, \mathbf{s}(V_{\mathrm{m}})) + I_{\mathrm{ext}}/(\chi C_m).$$

The solver approximates, cell by cell,

$$S_i'(V_{m,k}) \;\approx\; -\frac{I_{\mathrm{ion},i}(V_{m,k}+\varepsilon_d) - I_{\mathrm{ion},i}(V_{m,k})}{\varepsilon_d}, \qquad \varepsilon_d = \texttt{diagonalIionEpsilon}. \tag{33}$$

The linearised residual $\mathbf{A}\mathsf{x} - \theta S(\mathsf{x})$ gives the system

$$\big(\mathbf{A} - \theta\,\mathrm{diag}(S_k')\big)\,\mathsf{x}_{k+1} = \mathsf{r}\big(I_{\mathrm{ion}}(\mathsf{x}_k)\big) - \theta\,\mathrm{diag}(S_k')\,\mathsf{x}_k. \tag{34}$$

The update can still be relaxed with `nonlinearRelaxation`, and the same convergence and rollback logic as Picard is used.

Advantages. It captures the dominant local stiffness of $I_{\mathrm{ion}}(V_{\mathrm{m}})$ without building a full Jacobian. It often reduces the number of nonlinear iterations relative to Picard while preserving a sparse scalar PDE matrix.

Disadvantages. Only the cell-local diagonal sensitivity is included. Couplings through ODE states, high-order quadrature averaging and spatial reconstruction are not represented in the matrix. The finite-difference step `diagonalIionEpsilon` must be chosen above ODE/noise levels but small enough to be a useful derivative.

6.4 Jacobian-Free Newton–Krylov

JFNK applies Newton to (25):

$$\mathbf{J}(\mathsf{x}_k)\delta_k = -\mathsf{F}(\mathsf{x}_k), \qquad \mathsf{x}_{k+1} = \mathsf{x}_k + \alpha_k\delta_k,$$

but never assembles $\mathbf{J}$. GMRES sees only matrix-vector products approximated by

$$\mathbf{J}(\mathsf{x}_k)\,\mathbf{v} \;\approx\; \frac{\mathsf{F}(\mathsf{x}_k + \varepsilon\,\mathbf{v}) - \mathsf{F}(\mathsf{x}_k)}{\varepsilon}, \qquad \varepsilon \;=\; \varepsilon_0\,\frac{1 + \|\mathsf{x}_k\|_2}{\|\mathbf{v}\|_2}, \tag{35}$$

with $\varepsilon_0 = \texttt{jfnkEpsilon}$. Every evaluation of F re-runs the ODEs and recomputes I_{ion} from the same time-step-start state snapshot. This is expensive but gives a genuinely coupled nonlinear residual.

6.5 JFNK inner Krylov

The Newton linear system $\mathcal{J}_k\delta = -\mathsf{F}_k$ is solved with a back-end chosen by `jfnkLinearSolverBackend`:

- **Eigen** — `solveGMRES` implements GMRES(m) with modified Gram–Schmidt, using at most `jfnkMaxKrylovIterations` basis vectors per restart and `jfnkMaxRestarts` restarts.
- **PETSc** — `KSPGMRES` over a `MatShell` whose matrix-vector product is the finite-difference wrapper; the KSP object is persistent across steps.

In both cases the stopping test is controlled by `jfnkLinearTolerance` and the method is matrix-free: the linear operator is the finite-difference shell of (35), not a stored sparse matrix.

Preconditioner (optional). With `jfnkPreconditioner=diffusion` a right preconditioner $\mathbf{M}^{-1}$ is applied with $\mathbf{M} = \mathbf{A}$ (the linear, diffusive, SPD part of the Jacobian), factorised once as an incomplete ILU and reused; each Krylov step then costs a cheap *triangular solve* instead of an ODE integration. The default `none` reproduces the unpreconditioned path. See Section E.2.

Advantages. JFNK sees the full nonlinear dependence $V_\mathrm{m} \mapsto \mathbf{s}(V_\mathrm{m}) \mapsto I_\mathrm{ion}(V_\mathrm{m}, \mathbf{s})$ and usually has much better nonlinear convergence than Picard in stiff regimes. It also avoids explicitly forming a dense or complicated Jacobian.

Disadvantages. Every Krylov vector requires an extra residual evaluation, and hence a fresh ODE integration in every cell or quadrature point. The method is sensitive to the finite-difference perturbation, to the ODE tolerances and to the line-search choices. Without a preconditioner (`jfnkPreconditioner=none`) the number of matvecs — and therefore of ODE integrations — can be high; the diffusion preconditioner (Section 6.5) reduces it markedly. Empirically, for the benchmark of this report JFNK turns out to be considerably more expensive than Picard even with a preconditioner (Section F).

6.6 Armijo backtracking line search

The raw Newton update may increase the nonlinear residual when the linear model is poor. If `jfnkLineSearch=true`, the solver tries

$$\mathsf{x}_{k+1} = \mathsf{x}_k + \alpha\delta_k, \qquad \alpha = \texttt{nonlinearRelaxation},\ \alpha/2,\ \alpha/4, \ldots$$

until the Armijo sufficient-decrease condition holds:

$$\|\mathsf{F}(\mathsf{x}_k + \alpha\delta_k)\|_2^2 \le (1 - 2c\alpha)\ \|\mathsf{F}(\mathsf{x}_k)\|_2^2\,, \qquad c = \texttt{jfnkArmijoC}. \tag{36}$$

Backtracking stops after `jfnkLineSearchMaxIter` attempts or when the step length reaches `jfnkLineSearchAlphaMin`. This globalises Newton: large Newton steps are accepted when they reduce the residual, otherwise they are damped.

6.7 Initial guess, ODE clamp and rollback

The first guess for the time step is either V^n or the linear extrapolation

$$\mathsf{x}_0 = \mathsf{V}^n + \frac{\Delta t}{\Delta t_\mathrm{prev}}\left(\mathsf{V}^n - \mathsf{V}^{n-1}\right), \tag{37}$$

selected by `jfnkInitGuessOrder`. It is clamped to [`jfnkInitGuessVmMin`, `jfnkInitGuessVmMax`]. The same bracket can also be applied to voltage values passed into the ODE solver through `jfnkClampODEInput`; this protects the ionic model during JFNK finite-difference probes.

If the nonlinear loop reaches its iteration budget without satisfying the convergence tests, `nonlinearAcceptUnconverged` decides whether the last iterate is committed. With the default `false`, the solver rolls back V_m, I_ion and all ionic states to the beginning of the time step. Rollback prevents a bad nonlinear solve from contaminating the remaining activation simulation.

7 Linear solver: Eigen and PETSc back-ends

Matrix assembly and storage ($\mathbf{M}$, $\mathbf{K}$, $\mathbf{L}$, $\mathbf{A}$) *always* use `Eigen::SparseMatrix`. Solving the linear system of the assembled branches (Picard / diagonalIion) is handled by the `solveSparseSystem` wrapper, with a back-end chosen by `linearSolverBackend`:

linearSolverBackend	linearSolver	Implementation
Eigen	SparseLU	Eigen::SparseLU (direct LU)
Eigen	BiCGSTAB	Eigen::BiCGSTAB+IncompleteLUT
PETSc	(mapped)	KSP+PC (e.g. gmres+ilu, preonly+lu, cg+gamg)

With the PETSc back-end, the Eigen matrix is copied to a PETSc `Mat` and the KSP/PC type is taken from `petscLinearKspType` / `petscLinearPcType` (the latter accepts `ilu`, `lu`, `jacobi`, `gamg`, `hypre`, ...). All of them share `tolerance` and `maxIterations`.

Right preconditioning (2026-10). In PETSc, GMRES and BiCGStab precondition on the left by default, and `tolerance` then applies to the *preconditioned* residual. With `ilu` at a large Δt the two differ by a factor ~ 4, and the nonlinear test, which measures the true residual, was never met. The assembled KSP now preconditions on the **right** (as the JFNK shell KSP already did), and `cg` monitors the unpreconditioned norm. It is set before `KSPSetFromOptions`, so a PETSc option can still override it.

Factorisation reuse. Since $\mathbf{A} = \mathbf{M}/\Delta t - \theta\mathbf{L}$ is constant as long as Δt does not change, the matrix is assembled once and the solver (PETSc KSP or Eigen factorisation) is kept for the whole run, refreshed only when needed according to an update policy {Rebuild, ValuesOnly, Reuse}. This avoids re-factorising at every nonlinear iteration (Section E.1).

JFNK. The matrix-free JFNK branch does not instantiate these assembled wrappers; it uses its own Krylov solver (Section 6.5) over the finite-difference Jacobian shell, with back-end `jfnkLinearSolverBackend` (Eigen or PETSc).

8 Ionic ODE integration

For each cell or cell QP, the TNNP ODE (3) is advanced by the solver's own integrator selected via the `solver` key: adaptive `RKF45` by default [6], `RK4` or `Euler`; any other name is a `FatalError`. Every RKF45 integration starts from `initialODEStep` (it used to inherit the last sub-step of the previous call, and the key had no effect). Within the step V_{m} is not frozen: it comes from the driver of Section 13.3. Keys read from `constant/timeIntegrationProperties`:

Key	Meaning	Default
`solver`	Concrete ODE integrator	`RKF45`
`absTol`	Absolute tolerance per state	10^{-9}
`relTol`	Relative tolerance per state	10^{-6}
`maxSteps`	Cap on sub-steps per outer call	10^{4}
`initialODEStep`	Initial sub-step (ms)	10^{-5}

`relTol` drives the noise floor of the matrix-free Jacobian: $\eta_{\mathsf{F}} \sim$ `relTol`. Tightening `relTol` buys a lower attainable outer tolerance at the cost of more RKF45 sub-steps per Newton iteration.

9 TNNP numerical protection

The master switch `TNNPNumericalProtection` (default `false`) gates a local guard invoked at every external entry into the embedded TNNP kernel (`calculateCurrent`, `solveODE_start`, `solveODE_end`, `derivatives`).

Key	Type	Default
`TNNPNumericalProtection`	Switch	`false`
`TNNPClampGates`	Switch	`true`
`TNNPClampVmForRates`	Switch	`true`
`TNNPLogNumericalProtection`	Switch	`true`
`TNNPConcentrationFloor`	scalar (mM)	10^{-12}
`TNNPVMinForRates`	scalar (mV)	-200
`TNNPVMaxForRates`	scalar (mV)	$+100$
`TNNPVoltageZeroEps`	scalar (mV)	10^{-8}
`TNNPMaxProtectionLogEntries`	label	10^6
`TNNPProtectionLog`	word	`TNNP_numericalProtection_summary.dat`

When protection is enabled, the guards applied in order are:

1. **Non-finite** V: `!std::isfinite(V)` $\rightarrow$ reset to -86.2 mV.
2. V **clamp for rates** (if `TNNPClampVmForRates`): clamp to [`TNNPVMinForRates`, `TNNPVMaxForRates`].
3. $V = 0$ **singularity avoidance**: push V off zero by `TNNPVoltageZeroEps` (sign preserving) when $|V|$ falls below that threshold.
4. **Concentration floor** on K_i, Na_i, Ca_i, Ca_{SR}: non-finite or below-floor entries are reset to `TNNPConcentrationFloor`.
5. **Gate clamp** on $Xr_1, Xr_2, Xs, m, h, j, d, f, fCa, s, r, g$ (if `TNNPClampGates`): non-finite to zero, clamp to $[0, 1]$.

The embedded implementation stores a capped correction count and emits a compact summary file at exit; it does not depend on external ionic-model logging utilities.

Why it stays off (2026-10). When on, in the E02 temporal study the counter reaches its cap of 10^6 and the BDF schemes lose their order: with CC states, BDF2/3/4 give 1.74/2.22/3.54 with the protection and 1.89/2.96/4.60 without it, and no run needed it to finish. Since 2026-10-16 the suite harness does not turn it on either (`TNNP_NUMERICAL_PROTECTION = False`).

10 Activation tracking and benchmarks

Activation time per cell. First-crossing of V_{thr} in a time step is recorded by linear interpolation:

$$w \;=\; \text{clip}\Big(\frac{V_{\text{thr}} - V^n}{V^{n+1} - V^n},\; 0, 1\Big), \qquad t_{\text{act}} \;=\; t^n + w\,\Delta t, \tag{38}$$

with a per-cell flag cleared after the first crossing so subsequent re-crossings are ignored.

Early termination on P8. When `stopAfterPointActivation = true`, the cell nearest `stopActivationPoint` (P8 in the Niederer benchmark) is monitored and `effectiveEndTime` is set to t_{act}+ `stopDelayAfterActivation` once its $t_{\text{act}} > 0$.

Diagonal sampling. `sampleIDW` performs inverse-distance-weighted interpolation at `nDiagonalSamples` points between `diagonalStart` and `diagonalEnd`, with $k =$ `sampleNeighbours`, $p =$ `samplePower`. Because a diagonal sample averages its k nearest cells and unactivated cells contribute $t_{\text{act}} = 0$, a sample watched *live* climbs from below as the front fills its neighbourhood; the final value (all k neighbours activated) is the correct one.

Live activation output. `writeActivationSamples` is called once before the time loop (so `diagonalActivationTime` and `P8_activationTime.dat` exist from $t = 0$ with every point pending) and again whenever the activated-cell count increases, overwriting the files. The run can therefore be monitored as the front advances; the driver exposes the live file in each run directory by a symbolic link that is replaced by a real copy at the end.

Activation-failure detection. When `abortOnActivationFailure = true` the solver stops early (cleanly, exit 0) if the activation fails, avoiding a full run to `endTime`. Two criteria, using the running activated-cell count N_a and the time $t_\star$ of the last new activation:

- **no ignition:** $N_a = 0$ and $t >$ `activationIgnitionTimeout`;
- **conduction block:** $N_a > 0$, the stop point is not yet reached, and $t - t_\star >$ `activationStallTime`.

The outcome is always recorded in `postProcessing/.../activationStatus.dat` as one of `OK`, `FAILED_NO_IGNITION`, `FAILED_CONDUCTION_BLOCK` or `INCOMPLETE` (with the final time and N_a). The driver reads this file and, on a failure, logs it to `activation_status_summary.dat` and continues with the next configuration rather than aborting the sweep.

11 Solver flow chart

The pseudo-code below follows the actual solver. The shared setup is common to all nonlinear methods; then one of the three method-specific blocks is executed according to `nonlinearMethod`.

Algorithm 1 Shared time-step setup and final commit/rollback.

Read dictionaries and assemble (once) $\mathbf{M}$, $\mathbf{K}$, $\mathbf{L} = \mathbf{K}/(\chi C_m)$.
Initialise the persistent linear-solver state (PETSc KSP / Eigen factorisation) and mark Δt as not assembled.
while $t^n < T$ **do**
    Save snapshots: V^n, I_{ion}^n, $\mathbf{s}^n$.
    if Δt changed with respect to the assembled one **then**
        (Re)build $\mathbf{A} = \mathbf{M}/\Delta t - \theta\mathbf{L}$ and $\mathbf{B} = \mathbf{M}/\Delta t + (1-\theta)\mathbf{L}$; mark the solver for rebuild.
    end if ▷ constant if Δt is fixed ⇒ they are reused (Phase B)
    Build old source $S^n = -I_{\text{ion}}^n + I_{\text{ext}}^n/(\chi C_m)$.
    if `jfnkInitGuessOrder` ≥ 1 and previous step exists **then**
        $\mathsf{x}_0 \leftarrow \mathsf{V}^n + (\Delta t/\Delta t_{\text{prev}})(\mathsf{V}^n - \mathsf{V}^{n-1})$.
        Clamp x_0 to $[$`jfnkInitGuessVmMin`, `jfnkInitGuessVmMax`$]$.
    else
        $\mathsf{x}_0 \leftarrow \mathsf{V}^n$.
    end if
    Run Algorithm 2, 3 or 4.
    if not converged and `nonlinearAcceptUnconverged` is false **then**
        Roll back $V_{\text{m}}, I_{\text{ion}}, \mathbf{s}$ to the saved t^n snapshots.
    else
        Commit the final nonlinear iterate.
    end if
    Update activation times and benchmark samples.
end while

Algorithm 2 Picard nonlinear solve.

Input: Initial candidate x_0, state snapshot $\mathbf{s}^n$.

for $k = 0, \ldots,$ `nonlinearIterations` $- 1$ **do**
  Store previous iterate x_{k-1}, $I_{\mathrm{ion},k-1}$, $\mathbf{s}_{k-1}$ for residuals.
  EVALUATENONLINEARFIELDS(x_k): ▷ full evaluation #1 reset states to $\mathbf{s}^n$, reconstruct V_m at Iion quadrature points if enabled, call `ionicModel->solveODE`, compute $I_{\mathrm{ion},k}$.
  Build RHS $\mathsf{r}(I_{\mathrm{ion},k})$ using (26).
  Solve $\mathbf{A}\,\mathsf{V}_* = \mathsf{r}(I_{\mathrm{ion},k})$ with the linear back-end, reusing the factorisation if $\mathbf{A}$ did not change (*Reuse*; Picard has a constant $\mathbf{A}$).
  Relax: $\mathsf{x}_{k+1} \leftarrow \mathsf{x}_k +$ `nonlinearRelaxation`$(\mathsf{V}_* - \mathsf{x}_k)$.
  EVALUATENONLINEARFIELDS(x_{k+1}) ▷ full evaluation #2 (for residuals)
  Compute R_cpl, R_{V_m}, R_{I_ion} and optional $R_\mathbf{s}$.
  if all requested criteria pass and $k + 1 \geq$ `minNonlinearIterations` **then**
    Mark converged and exit loop.
  end if
end for
Note: each iteration performs *two* full field evaluations (LRE reconstruction + ODE + quadrature), which dominate the per-step cost; the linear solve is a minor fraction.

Algorithm 3 `diagonalIion` nonlinear solve.

Input: Initial candidate x_0, state snapshot $\mathbf{s}^n$.

for $k = 0, \ldots,$ `nonlinearIterations` $- 1$ **do**
  Evaluate states and $I_{\mathrm{ion},k}$ at x_k exactly as in Picard.
  Approximate the local derivative $S'_k \approx -[I_\mathrm{ion}(\mathsf{x}_k + \varepsilon_d) - I_\mathrm{ion}(\mathsf{x}_k)]/\varepsilon_d$, with $\varepsilon_d =$ `diagonalIionEpsilon`.
  Assemble $\mathbf{A}_k = \mathbf{A} - \theta\,\mathrm{diag}(S'_k)$ ▷ only the diagonal changes; the pattern is kept
  Build corrected RHS $\mathsf{r}_k = \mathsf{r}(I_{\mathrm{ion},k}) - \theta\,\mathrm{diag}(S'_k)\mathsf{x}_k$.
  Solve $\mathbf{A}_k\mathsf{V}_* = \mathsf{r}_k$ refreshing only the matrix values (*ValuesOnly*); with `petscReusePreconditioner` the preconditioner is also reused across iterations.
  Relax, re-evaluate ODEs/I_ion, compute residuals and test convergence as in Picard.
end for

Algorithm 4 Jacobian-Free Newton–Krylov solve.

Input: Initial candidate x_0, state snapshot $\mathbf{s}^n$.

for $k = 0, \ldots,$ `nonlinearIterations` $- 1$ **do**
  Evaluate $\mathsf{F}_k = \mathsf{F}(\mathsf{x}_k)$; this resets states to $\mathbf{s}^n$, solves ODEs and recomputes I_{ion}.
  Compute residuals. If all requested criteria pass, mark converged and exit.
  Define matrix-free product $\mathcal{J}_k\mathbf{v} = [\mathsf{F}(\mathsf{x}_k + \varepsilon\mathbf{v}) - \mathsf{F}_k]/\varepsilon$, with ε from `jfnkEpsilon` ▷ each matvec = 1 evaluation of F = 1 ODE integration.
  if `jfnkPreconditioner` = `diffusion` **then**
    Prepare (once / when Δt changes) the incomplete ILU of $\mathbf{M} = \mathbf{A}$ as right preconditioner $\mathbf{M}^{-1}$.
  end if
  Solve $\mathcal{J}_k\delta_k = -\mathsf{F}_k$ by restarted GMRES (back-end `jfnkLinearSolverBackend`: Eigen's in-house GMRES or PETSc's `KSPGMRES`/`MatShell`), using `jfnkMaxKrylovIterations`, `jfnkMaxRestarts`, `jfnkLinearTolerance` and, where relevant, the preconditioner $\mathbf{M}^{-1}$.
  if `jfnkLineSearch` **then**
    Start $\alpha =$ `nonlinearRelaxation` and backtrack until Armijo (36) succeeds, the trial budget is exhausted, or `jfnkLineSearchAlphaMin` is reached.
  else
    $\alpha \leftarrow$ `nonlinearRelaxation`.
  end if
  $\mathsf{x}_{k+1} \leftarrow \mathsf{x}_k + \alpha\delta_k$.
  Re-evaluate $\mathsf{F}(\mathsf{x}_{k+1})$, update fields, compute residuals and test convergence.
end for

Legend. `Blue` = Python constant set in `run_convergence.py`; `green` = OpenFOAM dictionary key read by the solver. Constants and keys are mapped one-to-one in section 12.

12 Manufactured driver: `run_convergence.py`

The convergence script `/home/pablo/OpenFOAM/pablo-v2312/run/tutorials_electro/highOrderManufacturedFDAImp` generates an MMS case whose numerical path is intended to match this solver. The tables below describe the constants exposed there, the OpenFOAM dictionary entries they generate, and which part of the discrete equations they affect. The same dictionary keys are read by `highOrderElectroActivationFoamImplicitPETSc`; only problem-specific items such as exact fields versus TNNP/stimulus differ.

12.1 Geometry and parameter sweep

Python constant	OpenFOAM key	Affected equation / mechanism
`DIMENSIONS`	—	Selects 2D or 3D manufactured domain and the corresponding LRE stencils in `P_LEVELS_BY_DIM`.
`N_VALUES`	`blockMeshDict` cell counts	Spatial refinement. This changes FV/LRE discretisation error in the PDE, Iion quadrature and state reconstruction.
`DT_VALUES`	`controlDict.deltaT`	Outer PDE time step. Also sets the integration interval for every local ODE solve.
`END_TIME`	`controlDict.endTime`	Final time of the transient MMS test.

Python constant	OpenFOAM key	Affected equation / mechanism
IMPLICIT_SCHEMES	spatialIntegrationProperties.implicitScheme	Choice of θ in (13): backwardEuler ($\theta = 1$) or crankNicolson ($\theta = 1/2$).
MASS_MATRIX_VALUES	spatialIntegrationProperties.massMatrix	Chooses lumped or LRE-consistent $\mathbf{M}$ in $\mathbf{A} = \mathbf{M}/\Delta t - \theta\mathbf{L}$.
ACTIVE_HO_PAIRS_Vm_States	useHighOrder_Vm / useHighOrder_Iion + LRECoeffs_*	Tuple (Vm-level, Iion/states-level). Vm level affects diffusion and consistent mass; Iion/states level affects where ODEs are solved and where the nonlinear source is integrated.
P_LEVELS_BY_DIM	highOrderCoeffs	Maps p1,p2,p3 to LRE polynomial order N, stencil target Nn, radial weight exponent k, and maxStencilSize.

12.2 Nonlinear method and convergence controls

Python constant	OpenFOAM key	Affected equation / mechanism
NONLINEAR_METHOD	nonlinearMethod	Selects the nonlinear strategy for the PDE/source coupling: Picard, JFNK, or diagonalIion.
IMPLICIT_NONLINEAR_ITERATIONS	nonlinearIterations	Maximum number of nonlinear corrections per PDE time step.
IMPLICIT_MIN_NONLINEAR_ITERATIONS	minNonlinearIterations	Minimum number of corrections before a convergence test may stop the loop.
NONLINEAR_TOLERANCE	nonlinearTolerance	Coupled PDE residual tolerance R_{cpl}. Used directly by JFNK and diagonalIion; in the electro solver it is also reported for Picard.
NONLINEAR_VM_TOLERANCE	nonlinearVmTolerance	Relative correction of the PDE unknown V_{m}.
NONLINEAR_STATES_TOLERANCE	nonlinearStatesTolerance	Relative correction of all ODE states. In the MMS solver these are u_1, u_2, u_3; in this electro solver these are the TNNP state variables.
NONLINEAR_REQUIRE_STATES_CONVERGENCE	nonlinearRequireStatesConvergence	If true, the state residual is part of the stopping criterion.
NONLINEAR_ACCEPT_UNCONVERGED	nonlinearAcceptUnconverged	If false, a failed electro nonlinear solve rolls back $V_{\mathrm{m}}, I_{\mathrm{ion}}$ and states to t^n.
NONLINEAR_IION_TOLERANCE	nonlinearIionTolerance	Relative correction of the nonlinear source I_{ion}.

Python constant	OpenFOAM key	Affected equation / mechanism
NONLINEAR_RELAXATION	nonlinearRelaxation	Picard/diagonal mixing factor or initial Newton step length for JFNK/Armijo.

12.3 JFNK and diagonal-Iion controls

Python constant	OpenFOAM key	Affected equation / mechanism
JFNK_MAX_KRYLOV_ITERATIONS	jfnkMaxKrylovIterations	Restart length m of GMRES(m), controlling memory and Krylov approximation quality.
JFNK_MAX_RESTARTS	jfnkMaxRestarts	Number of times the GMRES basis may be discarded and rebuilt.
JFNK_LINEAR_TOLERANCE	jfnkLinearTolerance	Relative tolerance of the Newton correction solve $\mathbf{J}\delta = -\mathsf{F}$.
JFNK_EPSILON	jfnkEpsilon	Base perturbation for matrix-free Jacobian-vector products. It should be above the ODE/noise floor but small enough to approximate a derivative.
JFNK_INIT_GUESS_ORDER	jfnkInitGuessOrder	Chooses lagged or linearly extrapolated initial V_{m} for the nonlinear solve.
JFNK_INIT_GUESS_VM_MIN/MAX	jfnkInitGuessVmMin/Max	Clamp interval for extrapolated guess and, if enabled, ODE inputs.
JFNK_CLAMP_ODE_INPUT	jfnkClampODEInput	Protects the ODE solve from unphysical JFNK finite-difference probes.
JFNK_LINE_SEARCH	jfnkLineSearch	Turns Armijo backtracking on/off.
JFNK_LINE_SEARCH_MAX_ITER	jfnkLineSearchMaxIter	Maximum number of halvings of the Newton step.
JFNK_LINE_SEARCH_ALPHA_MIN	jfnkLineSearchAlphaMin	Floor for the Armijo step length.
JFNK_ARMIJO_C	jfnkArmijoC	Sufficient-decrease parameter in (36).
DIAGONAL_IION_EPSILON	diagonalIionEpsilon	Finite-difference step for the diagonal source derivative in diagonalIion.

12.4 Linear PDE solver

Python constant	OpenFOAM key	Affected equation / mechanism
LINEAR_SOLVER_BACKEND	linearSolverBackend	Chooses the assembled back-end: Eigen or PETSc (Section 7).
IMPLICIT_LINEAR_SOLVER	linearSolver	Sparse solver used by Picard and diagonalIion for $\mathbf{A}\mathbf{V} = \mathbf{r}$ (SparseLU/BiCGSTAB in Eigen; mapped to KSP/PC in PETSc). JFNK instead uses GMRES on the Jacobian-free shell.
PETSC_LINEAR_KSP_TYPE / PETSC_LINEAR_PC_TYPE	petscLinearKspType / petscLinearPcType	KSP and preconditioner type when the back-end is PETSc (gmres+ilu, cg+gamg, ...).
IMPLICIT_TOLERANCE	tolerance	Relative tolerance of the iterative linear solve.
IMPLICIT_MAX_ITERATIONS	maxIterations	Iteration cap of the iterative linear solve.

12.5 State ODE integrator

Python constant	OpenFOAM key	Affected equation / mechanism
STATE_ODE_SOLVER	stateODESolver / timeIntegrationProperties.solver	Local ODE integrator for MMS states or TNNP states.
STATE_ODE_ABS_TOL	stateODEAbsTol / timeIntegrationProperties.absTol	Absolute tolerance of the ODE solve.
STATE_ODE_REL_TOL	stateODERelTol / timeIntegrationProperties.relTol	Relative ODE tolerance. This controls the noise floor seen by JFNK finite differences.
STATE_ODE_INITIAL_STEP	stateODEInitialStep / initialODEStep	First RKF45 sub-step attempted inside one PDE time step.
STATE_ODE_MAX_STEPS	stateODEMaxSteps / maxSteps	Maximum ODE substeps per local solve.

12.6 LRE high-order parameters per p-level

P_LEVELS_BY_DIM[dim][p] gives, for each (dim, p): N (polynomial order), Nn (stencil neighbours), k (search radius), maxStencilSize (cap). The script writes these into a nested highOrderCoeffs{LRECoeffs_Vm{...}; LRECoeffs_Iion{...}} block whenever HO is enabled for either Vm or I_{ion}.

12.7 Diagnostics and run-control

Python constant	OpenFOAM key	Affected mechanism
WRITE_NONLINEAR_RESIDUALS	writeNonlinearResiduals	Residual log on/off.
KEEP_LOGS_ON_SUCCESS	—	Python-only switch: keep solver/blockMesh logs after successful runs. Failed-run logs are always kept.

For the electro-activation tutorial the analogous driver also exposes activation-threshold, P8 stopping and diagonal-sampling parameters. They do not affect the PDE/ODE solve itself; they affect only stopping and post-processing of activation times.

13 October 2026 changes and new results

This section collects what changed in the solver between August and 2026-10-18, and the verification-suite tests that measure it (`run/tutorials_electro/electroTestParallelVerification_2D/` and, for the MMS, `mmsTestParallelVerification_2D/`). The earlier sections have already been corrected where they had become false; the detail is here. The results of Section 14 **predate** all of this.

13.1 Code shared with the MMS solver

What this solver shares with `highOrderManufacturedFDAImplicitPETScDistributed` moved to headers in `src/monodomainCommon/`:

Header	Contents
`monodomainBase.H`	global row offset, option validation, nonlinear stopping test
`monodomainAssembly.H`	assembly of $\mathbf{M}$ and $\mathbf{K}$ (orthogonal and high order)
`monodomainPetsc.H`	distributed matrices, cached KSP, KSP/PC selection
`monodomainKrylov.H`	in-house GMRES and JFNK
`monodomainTimeSchemes.H`	θ/BDF coefficients, ESDIRK3 tableau, V_{m} driver
`monodomainReconstruction.H`	reconstructions to the Gauss points
`openMPControls.H`, `jfnkPreconditionerControls.H`	OpenMP keys and JFNK preconditioner keys

They are not a library: they are included *inside* the anonymous namespace of each `.C`, so each executable compiles its own copy and a change there changes both solvers. The code walkthrough (`codigo_*.pdf`, generated by `mkcodereport.py`) looks up each function in the `.C` and in those headers.

13.2 BDF1–4 with an ESDIRK3 startup

Ported from the MMS solver. A BDF-k step reads

$$\Big(\frac{a_0}{\Delta t}\mathbf{M}-\mathbf{L}\Big)\mathsf{V}^{n+1} \;=\; \frac{1}{\Delta t}\mathbf{M}\,\mathsf{H}^{n}+\mathbf{s}^{n+1}, \qquad \mathsf{H}^{n}=-\sum_{j=1}^{k} a_j\,\mathsf{V}^{n+1-j}, \tag{39}$$

i.e. the same pair of operators as the θ scheme with $\theta=1$ and a different scalar in front of $\mathbf{M}$: assembly, sparsity and the cached factorisation are unchanged, and the history enters through a single product with $\mathbf{M}$. BDF3 and BDF4 are $A(86°)$- and $A(73°)$-stable; BDF5 and BDF6 are not offered (52° and 18°, which the reaction Jacobian of a front does not respect).

The steps a BDF-k cannot take by itself — the first $k-1$, the $k-1$ after a history restart, and a step of non-nominal length (the clipped last step, the stop point) — are taken with **ESDIRK3(2)4L[2]SA**, a one-step, L-stable method, third order with respect to the semi-discrete system, which is what preserves the nominal order (`bdfStartup ESDIRK3`, the only value).

The history is **restarted** when the stimulus changes (a jump in the forcing that a multistep method cannot cross), after a rolled-back step, and on a step of non-nominal length. The stimulus is also **averaged over the step**: the injected charge is exact for any Δt, whereas before the stimulus lasted one step too long.

13.3 The V_{m} driver of the ODEs

The states are integrated with sub-steps inside the PDE step, and each sub-step needs V_{m} at an intermediate instant. With V_{m} frozen at the candidate, TNNP gave order 1 even with Crank–Nicolson. The ODE now takes it from a Lagrange polynomial in time (`interpolateVmDriver`):

- `stateODEDriver linear`: the line between V^n and the candidate; second order, it caps every BDF at 2 (MMS M21: BDF4 2.04).
- `stateODEDriver history` (default): it also goes through the BDF history levels or the ESDIRK stage values, which the scheme already stores; no extra evaluations (M21: BDF4 3.89).

`backwardEuler`, `crankNicolson` and `BDF1` store no level older than V^n and use the line under either value.

13.4 The V_m history at the Gauss points

On the Gauss-point paths (`gaussPointODE` and the hybrid's front) the driver was always the line, because raising it seemed to require storing every level at every point. With `stateODEGaussPointHistory true` those paths use the same polynomial as the cell path *without storing anything between steps*: each history level (or each accepted ESDIRK stage value) is reconstructed at the Gauss points at the start of the step, with the same operator that gives V^n there (`reconstructVmLevelAtIionIntegrationPoints`). Within the step $k-1$ temporary arrays of Gauss-point size live. It only acts with `stateODEDriver history`; the solver has it off by default and the harness turns it on.

Without the key, BDF3 and BDF4 stayed at ∼2.4–2.5 on GP (E02); with it they reach their order (Section 13.8). E36 measures the cost, BDF4 at 0.25 mm, $\Delta t = 2.5 \times 10^{-5}$ s, 4 ranks, two alternated repetitions:

Mode	loop without the key [s]	loop with the key [s]	cost
GP	51.8	56.4	+8.9 %
CC + hybrid	12.6	13.2	+5.1 %

Peak memory does not change measurably (173 MB); the theoretical temporary memory is 24 bytes per Gauss point with BDF4, against 152 for the 19 TNNP states at that same point.

13.5 The hybrid's temporal floor, and its fix

With the corrections above, pure CC and GP converged in Δt, but the hybrid sat on a **floor** of $\sim 2.2 \times 10^{-3}$ in the Cauchy difference that no scheme got below (all equal on the finest pair).

What it was not. The first hypothesis was cells near the `hybridFrontVmRange` threshold entering and leaving the band. The mode-switch diagnostic ruled it out: ∼ 250 switches per run, constant as Δt shrinks. A hysteresis in the classification (`hybridFrontVmRangeExit`) did not help either; without the limiter the runs hit `SIGFPE`, and it was removed from the solver. Widening the band did not help.

What it was. The smooth cells rebuilt their cell state every step as the average of its reconstruction at the Gauss points. Averaging a reconstruction is not the identity: for a quadratic reconstruction on a square cell of side h,

$$\frac{1}{|\Omega_i|}\int_{\Omega_i} \widehat{s}\,\mathrm{d}\mathbf{x} \;=\; s_i + \frac{h^2}{24}\,\Delta s + O(h^4), \tag{40}$$

a diffusion of the states applied *once per step*, hence with an effective coefficient $\propto h^2/(24\,\Delta t)$: stronger the smaller Δt, which is exactly a floor in temporal convergence.

The fix. The smooth cells keep their cell state, `hybridCellStates`, between steps, as pure CC does, and only the front lives at the Gauss points; the Gauss average is used once, when a cell leaves the band. Without the limiter, CN went from 1.49×10^{-3} to 3.72×10^{-4} (order 2.00) and BDF4 from 2.80×10^{-4} to 5.8×10^{-6} on the finest pair: the same as GP with history, at half the cost.

Diagnostic in the log. At the end of every run with the hybrid the solver prints `Hybrid front band: N mode switches; M cells switched more than twice (max K)`. Without the hybrid it prints nothing.

13.6 Other changes

- **Common stopping test with a floor exit** (stopping-test paragraph of the nonlinear-solver section): the three methods and the ESDIRK stages use `nonlinearIterationExit`.
- **Right preconditioning** of the assembled KSP, so that the linear tolerance measures the true residual (Section 7).
- **Reconstruction of K fields at once** (`reconstructCellFieldsAtIntegrationPoints`): the MLS stencil is walked once per cell for the 19 states; bit-identical to reconstructing them one at a time.
- **Dirichlet refused.** `fixedValue` and `fixedVoltage` are refused at start-up: the operator assembled the diagonal term but the right-hand-side lift was never formed, so the boundary was clamped to zero. Measured with walls at -40 mV on the low-order path: 47 mV of error and growing, exit 0, no warning. The accepted conditions are `zeroGradient`, `empty` and the processor patches.
- **OpenMP**: the reconstructions to the Gauss points are threaded, and `Eigen::setNbThreads` ties Eigen's products to the same key.

13.7 Defaults: solver and harness

Key	solver	suite harness (since 2026-10-16)
`TNNPNumericalProtection`	`false`	`False`
`stateReconstructLimiter`	`false`	`False`
`stateODEGaussPointHistory`	`false`	`True`
`stateODEDriver`	`history`	`history`
`bdfStartup`	`ESDIRK3`	`ESDIRK3`

13.8 New results

E02: temporal convergence per scheme and state mode. 2-D, 0.5 mm, V_m at $t = 10$ ms, Cauchy in $\Delta t = 10^{-4} \ldots 6.25 \times 10^{-6}$ s, six schemes and four modes, Gauss history on, no protection, no limiter (120 runs). Observed order on the finest pair:

Scheme	nominal	low order (NO,NO,NO)	pure CC (p3,p3,p3)	CC + hybrid (p3,p3,p3)	pure GP (p3,NO,p3)
`backwardEuler`, `BDF1`	1	0.59	0.82	0.97	0.97
`crankNicolson`	2	1.95	2.00	2.00	2.00
`BDF2`	2	1.75	1.89	1.59	1.59
`BDF3`	3	2.77	2.96	2.93	2.93
`BDF4`	4	4.60	4.60	5.59	5.47
mean time per run [s]		148	185	568	1135

`BDF1` gives the same V_m as `backwardEuler` in every case. The verdict is FAIL (12/13) because of a single check: in pure CC, backwardEuler and BDF1 do not shrink the difference on the coarsest pair (2.23×10^{-2} then 2.47×10^{-2}), saturation of an L2 measure of a front displaced by more than its width. Still open: BDF2 at $\sim$1.59 on the Gauss-point paths, and backwardEuler at 0.59 on low order.

E35: spatial (Cauchy) convergence of the activation time at P8. With high order T_{P8} must rise under refinement. GP and CC + hybrid do (33.68, 41.49, 44.11, 44.87 ms at 0.5, 0.25, 0.125 and 0.0625 mm, equal to each other, the hybrid 3–6 times cheaper); pure CC is not monotone (47.39, 50.82, 48.61, 45.56 ms): it reconstructs the states, nearly discontinuous at the front, and loses the high order there. Low order falls (156.7, 66.3, 49.5, 45.8, 45.0 ms). This campaign **predates** the hybrid fix and the current harness defaults, and was left with 8 pending runs at 0.03125 mm.

E36: cost of the Gauss history. PASS, see Section 13.4.

M02 (MMS, exact solution): GP with history. 2-D hexa $N = 40$, p3. With the Gauss-history key the GP path gives, by route A (Cauchy) and route B (error against the analytical solution): CN 2.000/2.000, BDF2 1.979/1.992, BDF3 2.932/2.971, BDF4 3.891/3.908. Without the key, by route B, BDF4 stalls at an error of $\sim 2 \times 10^{-10}$. This is the exact-solution check that the key brings the Gauss path to its nominal order.

14 Results

Note (2026-10). The tables in this section were run with the July–August solver, **before** all the changes of Section 13 (among others: θ scheme only, state limiter on by default, and the hybrid with the temporal floor of Section 13.5). They are kept as a record and were not re-run; the results with the current solver are in Section 13.8.

This section collects the current results for the 2D and 3D Niederer-style benchmarks. The 2D benchmark (Section 14.1) was re-run with an extended `Picard` sweep covering the state-integration modes and the front-aware hybrid introduced in Section 5; the 3D benchmark (Section 14.2) keeps the previously reported results (`Picard` and `diagonalIion`) and is not modified here. All runs use the Crank–Nicolson time scheme ($\theta = 1/2$), the high-order consistent mass matrix when LRE is enabled, and the `RKF45` state-ODE integrator. Wall-clock time and peak RSS are defined in Section 14.1.1, together with the hardware-independent cost quantities.

14.1 2D Niederer-style benchmark

The 2D case is the planar 20×8 mm reduction (a single cell in z, planar fibres). The sweep reported here is exhaustive in `Picard` and lives under `highOrderNiedererEtAl2012ImplicitPETSc/results/example0/2D_run_20260705_110250/`. It covers the product of:

- **Mesh:** structured hexahedral (`hexa`) and unstructured triangular (`triangularUnstr`).
- **Hybrid:** `frontAwareHybrid false` (8 configurations) and `true` (4 configurations).
- **Time step:** $\Delta t \in \{10^{-4}, 10^{-5}, 10^{-6}\}$ s.
- **Spatial mesh:** $\Delta x \in \{0.5, 0.25, 0.125, 0.1, 0.05\}$ mm. The two finest levels (0.1 and 0.05 mm) were run only for the configurations that reach the converged regime; combinations that were not run appear as `--` in the tables.

Each configuration is identified by the triplet `(Vm, states, Iion)`:

- `Vm`$\in$ {`NO`,`p3`}: order of the LRE reconstruction of the potential in the diffusion operator (`NO` = classical finite volumes).
- `states`$\in$ {`na`,`GP`,`CCp1`,`CCp3`}: state integration mode (Section 5). `na` = not applicable (base case without high order in I_{ion}); `GP` = `gaussPointODE` (ODE at every Gauss point); `CCp`k = `cellCentredReconstruct` (ODE at the cell centre + LRE reconstruction of order k of the states to the Gauss points).
- `Iion`$\in$ {`NO`,`p1`,`p3`}: order of the LRE reconstruction at the ionic quadrature points.

When `frontAwareHybrid` is active, the `CCp`k configurations solve the ODE at the Gauss points only in the front cells and by `cellCentredReconstruct` everywhere else (Section 5.2).

14.1.1 Computational cost quantities

The run time of a simulation depends on which other processes shared the machine, so it is not a clean measure of the cost of the algorithm (see the thread-contention analysis in the associated comparison note). To be able to compare configurations reproducibly, each table reports, besides time and memory, two *deterministic* quantities (independent of the hardware and of the machine load):

wall [min] *Wall-clock time*: elapsed solver time (`setup+loop`) reported by its internal timer, rounded to minutes. It is what the run "takes", but it **depends on the machine load** (CPU contention, thread oversubscription).

CPU [min] *CPU time* (`User time` from `/usr/bin/time -v`): sum of CPU-seconds consumed by the process in user mode across all threads. It is less sensitive to load than the wall time, but it still inflates when threads *busy-wait* under contention; that is why CPU > wall in the multi-threaded runs (e.g. at $\Delta x = 0.125\,\mathrm{mm}$).

RSS [MiB] *Peak resident set size*: maximum resident memory of the process, $\lfloor \mathrm{kB}/1024 \rfloor$. It scales with the LRE stencil size and with the number of cells.

N_{nl} *Total nonlinear iterations*: sum over all time steps of the number of `Picard` iterations. It equals the number of full nonlinear residual evaluations — each one integrates the ODEs, evaluates I_{ion} and assembles/solves the linear PDE. It is **deterministic** (given Δt, the mesh and the tolerances) and therefore the cleanest cost reference across configurations.

N_{lin} *Total linear iterations*: sum of the GMRES iterations across all linear PDE solves. Deterministic; it measures the work of the linear solver.

Important warning about N_{nl} and N_{lin}. These two count iterations of the *solver*, not the cost of integrating the ODE at each evaluation. That per-evaluation cost differs strongly between modes: `gaussPointODE` integrates N_q stiff ODEs per cell (one per Gauss point), whereas `cellCentredReconstruct` and the smooth cells of the hybrid integrate *one* per cell. This is why two runs with the same N_{nl} can differ several-fold in wall/CPU (see the hybrid discussion below): the wall and CPU times capture the cost of integrating the ODEs, whereas N_{nl} and N_{lin} capture the number of solves. The four quantities are complementary.

The number of time steps n_{steps} (also deterministic) is not tabulated separately: for $\Delta t \le 10^{-5}$ s `Picard` typically converges in one iteration per step, so that $N_{\mathrm{nl}} \approx n_{\mathrm{steps}}$; at $\Delta t = 10^{-4}$ s several iterations per step are needed and $N_{\mathrm{nl}} > n_{\mathrm{steps}}$. Runs stop as soon as P8 (the far corner) activates, so $n_{\mathrm{steps}} \approx t_{\mathrm{P8}}/\Delta t$; the runs marked FAIL are aborted at $t = 0.02\,\mathrm{s}$ (ignition timeout).

14.1.2 Structured hexahedral mesh, no hybrid (`frontAwareHybrid false`)

Table 7: Per-case performance — structured hexahedral mesh, no hybrid, all `Picard`. For each triplet (`Vm, states, Iion`) the rows list the available Δx [mm] and the columns the three Δt, grouped three metrics at a time. FAIL = no ignition (the run is aborted at $t = 0.02\,\mathrm{s}$ by the failure detector); a P8 value implies full activation of the domain; `--` = combination not run. N_{nl} and N_{lin} are the totals of nonlinear and linear iterations (deterministic); wall and CPU in minutes; RSS in MiB.

Δt[s]	10^{-4}	10^{-5}	10^{-6}	10^{-4}	10^{-5}	10^{-6}	10^{-4}	10^{-5}	10^{-6}
(Vm, states, Iion) = (NO, na, NO)									
Δx[mm]	P8 [ms]			N_{nl}			N_{lin}		
0.5	153.32	157.15	157.31	6 293	16 136	161 519	18 879	32 272	323 038
0.25	62.99	67.30	67.42	3 510	6 854	68 652	12 748	13 708	137 304
0.125	45.03	50.05	50.26	2 237	5 078	50 971	11 183	15 233	101 942
0.1	42.76	47.86	48.11	2 067	4 851	48 751	11 573	14 553	97 502
0.05	40.22	45.28	45.60	1 618	4 583	46 136	14 560	18 331	138 407

Δt[s]	10^{-4}	10^{-5}	10^{-6}	10^{-4}	10^{-5}	10^{-6}	10^{-4}	10^{-5}	10^{-6}
Δx[mm]		wall [min]			CPU [min]			RSS [MiB]	
0.5	13	27	66	13	27	66	181	181	181
0.25	27	46	115	27	46	115	187	188	189
0.125	69	131	339	106	208	1098	213	212	212
0.1	104	201	539	150	302	1546	232	232	232
0.05	325	739	1855	362	842	2835	393	395	388
(Vm, states, Iion) = (NO, GP, p1)									
Δx[mm]		P8 [ms]			N_{nl}			N_{lin}	
0.5	91.05	94.34	94.48	4 312	9 722	97 348	12 936	19 444	194 696
0.25	54.86	58.85	58.95	2 829	5 998	60 074	10 126	11 996	120 148
0.125	43.21	48.04	48.27	1 886	4 875	48 965	9 430	14 625	97 930
0.1	41.84	46.79	47.06	1 690	4 743	47 688	9 406	14 229	95 376
0.05	40.14	45.18	45.50	1 611	4 573	46 036	14 497	18 290	138 108
Δx[mm]		wall [min]			CPU [min]			RSS [MiB]	
0.5	18	35	78	18	35	78	181	182	181
0.25	47	85	192	47	85	192	191	193	192
0.125	114	254	603	144	334	1322	228	228	229
0.1	174	406	996	211	503	1975	258	258	257
0.05	656	1535	3652	694	1637	4613	490	492	490
(Vm, states, Iion) = (NO, CCp1, p1)									
Δx[mm]		P8 [ms]			N_{nl}			N_{lin}	
0.5	80.23	FAIL	FAIL	4 079	2 004	20 002	12 237	4 008	40 004
0.25	60.37	FAIL	FAIL	3 428	2 004	20 002	11 872	4 008	40 004
0.125	49.71	FAIL	FAIL	2 470	2 003	20 002	12 350	6 009	40 004
0.1	48.06	FAIL	FAIL	2 284	2 003	20 002	12 225	6 009	40 004
0.05	45.95	FAIL	FAIL	1 845	2 003	20 002	16 603	8 012	60 006
Δx[mm]		wall [min]			CPU [min]			RSS [MiB]	
0.5	11	6	12	11	6	12	183	183	183
0.25	36	23	48	36	23	48	194	196	196
0.125	94	80	185	136	111	485	241	241	241
0.1	148	135	320	199	177	727	277	276	275
0.05	473	530	1230	516	575	1644	567	565	564
(Vm, states, Iion) = (p3, GP, NO)									
Δx[mm]		P8 [ms]			N_{nl}			N_{lin}	
0.5	180.55	184.69	184.90	7 027	18 725	187 450	21 081	56 175	562 350
0.25	68.84	73.57	73.73	4 072	7 453	74 677	12 216	22 359	224 031
0.125	48.83	54.36	54.58	2 688	5 503	55 232	8 064	16 437	165 696
Δx[mm]		wall [min]			CPU [min]			RSS [MiB]	
0.5	15	38	169	60	211	2166	192	192	188
0.25	31	50	127	58	109	596	227	228	223
0.125	81	144	398	126	237	1357	373	373	373
(Vm, states, Iion) = (p3, GP, p1)									
Δx[mm]		P8 [ms]			N_{nl}			N_{lin}	
0.5	68.92	71.41	71.53	4 056	9 550	95 641	12 168	28 650	286 923
0.25	58.81	63.12	63.27	3 120	6 391	64 047	9 360	19 173	192 141
0.125	46.33	51.39	51.61	2 216	5 204	52 246	6 648	15 599	156 738
Δx[mm]		wall [min]			CPU [min]			RSS [MiB]	
0.5	20	38	138	69	155	1295	193	193	193
0.25	52	96	260	91	174	1038	231	231	231
0.125	132	273	689	170	366	1581	385	385	383
(Vm, states, Iion) = (p3, GP, p3)									
Δx[mm]		P8 [ms]			N_{nl}			N_{lin}	
0.5	33.09	36.31	36.56	1 129	3 717	37 404	3 387	11 151	112 212
0.25	40.22	44.20	44.43	1 369	4 494	45 160	4 107	13 482	135 480
0.125	40.84	45.40	45.66	1 392	4 604	46 293	4 176	13 812	138 879
Δx[mm]		wall [min]			CPU [min]			RSS [MiB]	
0.5	21	56	141	35	101	595	198	199	199
0.25	96	255	622	113	310	1171	252	252	252
0.125	335	932	2228	359	1012	3026	467	461	463
(Vm, states, Iion) = (p3, CCp1, p1)									
Δx[mm]		P8 [ms]			N_{nl}			N_{lin}	

Δt[s]	10^{-4}	10^{-5}	10^{-6}	10^{-4}	10^{-5}	10^{-6}	10^{-4}	10^{-5}	10^{-6}
0.5	91.40	FAIL	FAIL	4 636	2 004	20 002	13 908	6 012	60 006
0.25	66.46	FAIL	FAIL	4 168	2 004	20 002	12 504	6 012	60 006
0.125	52.98	FAIL	FAIL	2 925	2 003	20 002	8 775	6 009	60 006
0.1	50.47	FAIL	FAIL	2 554	2 003	20 002	10 214	6 009	60 006
0.05	46.51	FAIL	FAIL	1 879	2 003	20 002	9 393	6 009	60 006
Δx[mm]		wall [min]			CPU [min]			RSS [MiB]	
0.5	15	7	24	71	31	267	196	195	194
0.25	46	24	62	96	48	304	237	236	238
0.125	111	82	201	161	116	544	413	413	413
0.1	160	133	322	217	175	739	538	534	533
0.05	470	523	1270	513	569	1717	1433	1416	1416
`(Vm, states, Iion) = (p3, CCp3, p3)`									
Δx[mm]		P8 [ms]			N_{nl}			N_{lin}	
0.5	66.63	FAIL	FAIL	2 788	2 004	20 002	8 364	6 012	60 006
0.25	60.80	FAIL	FAIL	2 942	2 004	20 002	8 826	6 012	60 006
0.125	52.20	FAIL	FAIL	2 455	2 003	20 002	7 365	6 009	60 006
0.1	50.15	FAIL	–	2 231	2 003	–	8 922	6 009	–
0.05	46.48	–	–	1 863	–	–	9 313	–	–
Δx[mm]		wall [min]			CPU [min]			RSS [MiB]	
0.5	10	8	33	44	32	273	203	203	203
0.25	37	28	101	73	52	344	271	270	271
0.125	107	97	374	149	131	752	541	538	538
0.1	163	161	–	211	202	–	743	741	–
0.05	540	–	–	581	–	–	2416	–	–

14.1.3 Structured hexahedral mesh, front-aware hybrid (`frontAwareHybrid true`)

Table 8: Per-case performance — structured hexahedral mesh, front-aware hybrid, all `Picard`. For each triplet `(Vm, states, Iion)` the rows list the available Δx [mm] and the columns the three Δt, grouped three metrics at a time. FAIL = no ignition (the run is aborted at $t = 0.02$ s by the failure detector); a P8 value implies full activation of the domain; `--` = combination not run. N_{nl} and N_{lin} are the totals of nonlinear and linear iterations (deterministic); wall and CPU in minutes; RSS in MiB.

Δt[s]	10^{-4}	10^{-5}	10^{-6}	10^{-4}	10^{-5}	10^{-6}	10^{-4}	10^{-5}	10^{-6}
`(Vm, states, Iion) = (NO, na, NO)`									
Δx[mm]		P8 [ms]			N_{nl}			N_{lin}	
0.5	153.32	157.15	157.31	6 293	16 136	161 519	18 879	32 272	323 038
0.25	62.99	67.30	67.42	3 510	6 854	68 652	12 748	13 708	137 304
0.125	45.03	50.05	50.26	2 237	5 078	50 971	11 183	15 233	101 942
Δx[mm]		wall [min]			CPU [min]			RSS [MiB]	
0.5	13	27	69	13	27	69	181	181	181
0.25	28	47	117	28	47	117	188	189	188
0.125	69	131	341	108	211	1119	212	212	212
`(Vm, states, Iion) = (NO, CCp1, p1)`									
Δx[mm]		P8 [ms]			N_{nl}			N_{lin}	
0.5	91.05	94.34	94.48	4 320	9 722	97 350	12 960	19 444	194 699
0.25	54.86	58.85	58.95	2 826	5 998	60 076	10 117	11 996	120 152
0.125	43.20	48.05	48.29	1 904	4 876	48 988	9 519	14 627	97 976
0.1	41.85	46.79	47.07	1 695	4 744	47 702	9 430	14 232	95 403
0.05	40.14	45.18	45.51	1 611	4 573	46 040	14 497	18 291	138 119
Δx[mm]		wall [min]			CPU [min]			RSS [MiB]	
0.5	12	23	57	12	23	57	182	183	183
0.25	27	50	126	27	50	126	195	196	195
0.125	63	136	385	96	213	1136	240	241	240
0.1	94	220	630	132	319	1609	278	279	277

Δt[s]	10^{-4}	10^{-5}	10^{-6}	10^{-4}	10^{-5}	10^{-6}	10^{-4}	10^{-5}	10^{-6}
0.05	340	825	2208	379	927	3175	564	568	570
`(Vm, states, Iion) = (p3, CCp1, p1)`									
Δx[mm]		P8 [ms]			N_{nl}			N_{lin}	
0.5	68.93	71.44	71.56	4 041	9 554	95 681	12 123	28 662	287 043
0.25	58.82	63.11	63.27	3 120	6 390	64 044	9 360	19 170	192 132
0.125	46.33	51.39	51.61	2 218	5 204	52 250	6 654	15 598	156 750
Δx[mm]		wall [min]			CPU [min]			RSS [MiB]	
0.5	13	27	114	62	143	1277	195	195	195
0.25	31	56	175	69	134	955	238	238	237
0.125	72	149	452	111	240	1371	414	414	414
`(Vm, states, Iion) = (p3, CCp3, p3)`									
Δx[mm]		P8 [ms]			N_{nl}			N_{lin}	
0.5	33.09	36.30	36.51	1 130	3 715	37 350	3 390	11 145	112 050
0.25	40.22	44.22	44.54	1 371	4 496	45 266	4 113	13 488	135 798
0.125	40.84	45.43	45.90	1 391	4 607	46 527	4 173	13 821	139 581
0.1	40.72	45.48	45.98	1 402	4 609	46 573	5 607	13 827	139 719
0.05	40.05	45.04	45.52	1 604	4 558	46 051	8 019	13 674	138 153
Δx[mm]		wall [min]			CPU [min]			RSS [MiB]	
0.5	6	17	64	20	62	519	202	202	202
0.25	20	60	216	37	115	767	267	267	267
0.125	64	205	845	89	298	1841	530	530	530
0.1	104	306	1178	135	403	2147	726	725	726
0.05	419	1123	4211	456	1224	5239	2370	2367	2351

14.1.4 Unstructured triangular mesh, no hybrid

Table 9: Per-case performance — unstructured triangular mesh, no hybrid, all `Picard`. For each triplet `(Vm, states, Iion)` the rows list the available Δx [mm] and the columns the three Δt, grouped three metrics at a time. FAIL = no ignition (the run is aborted at $t = 0.02$ s by the failure detector); a P8 value implies full activation of the domain; `--` = combination not run. N_{nl} and N_{lin} are the totals of nonlinear and linear iterations (deterministic); wall and CPU in minutes; RSS in MiB.

Δt[s]	10^{-4}	10^{-5}	10^{-6}	10^{-4}	10^{-5}	10^{-6}	10^{-4}	10^{-5}	10^{-6}
`(Vm, states, Iion) = (NO, na, NO)`									
Δx[mm]		P8 [ms]			N_{nl}			N_{lin}	
0.5	55.81	59.70	59.88	3 000	6 060	60 760	12 000	18 179	121 520
0.25	45.18	50.28	50.51	2 231	5 095	51 168	15 615	16 180	102 336
0.125	40.93	46.00	46.30	1 662	4 657	46 858	18 281	23 284	140 574
Δx[mm]		wall [min]			CPU [min]			RSS [MiB]	
0.5	15	25	61	15	25	61	183	183	183
0.25	45	88	222	72	150	848	197	197	198
0.125	128	304	808	166	407	1803	246	247	247
`(Vm, states, Iion) = (NO, GP, p1)`									
Δx[mm]		P8 [ms]			N_{nl}			N_{lin}	
0.5	55.81	59.70	59.88	3 000	6 060	60 760	12 000	18 179	121 520
0.25	45.18	50.28	50.51	2 231	5 095	51 168	15 615	16 180	102 336
0.125	40.93	46.00	46.30	1 662	4 657	46 858	18 281	23 284	140 574
Δx[mm]		wall [min]			CPU [min]			RSS [MiB]	
0.5	15	25	60	15	25	60	183	184	183
0.25	45	88	231	73	150	853	197	197	198
0.125	118	297	687	147	389	1482	249	248	250
`(Vm, states, Iion) = (NO, CCp1, p1)`									
Δx[mm]		P8 [ms]			N_{nl}			N_{lin}	
0.5	55.81	59.70	59.88	3 000	6 060	60 760	12 000	18 179	121 520
0.25	45.18	50.28	50.51	2 231	5 095	51 168	15 615	16 180	102 336

Δt[s]	10^{-4}	10^{-5}	10^{-6}	10^{-4}	10^{-5}	10^{-6}	10^{-4}	10^{-5}	10^{-6}
0.125	40.93	46.00	46.30	1 662	4 657	46 858	18 281	23 284	140 574
Δx[mm]		wall [min]			CPU [min]			RSS [MiB]	
0.5	14	25	64	14	25	64	185	185	185
0.25	43	86	243	71	149	867	203	203	204
0.125	131	320	874	169	421	1891	272	271	273
`(Vm, states, Iion) = (p3, GP, NO)`									
Δx[mm]		P8 [ms]			N_{nl}			N_{lin}	
0.5	75.42	57.19	57.29	4 320	7 956	79 678	12 960	23 868	239 034
0.25	51.56	56.76	56.99	2 981	5 741	57 626	10 672	17 223	172 878
0.125	43.26	48.42	48.66	2 144	4 904	49 263	10 719	14 712	147 789
Δx[mm]		wall [min]			CPU [min]			RSS [MiB]	
0.5	20	31	88	50	81	608	203	202	200
0.25	51	85	217	71	122	584	272	272	268
0.125	150	304	770	189	398	1645	561	560	562
`(Vm, states, Iion) = (p3, GP, p1)`									
Δx[mm]		P8 [ms]			N_{nl}			N_{lin}	
0.5	75.42	57.19	57.29	4 320	7 956	79 678	12 960	23 868	239 034
0.25	51.56	56.76	56.99	2 981	5 741	57 626	10 672	17 223	172 878
0.125	43.26	48.42	48.66	2 144	4 904	49 263	10 719	14 712	147 789
Δx[mm]		wall [min]			CPU [min]			RSS [MiB]	
0.5	23	37	124	75	134	1090	204	204	204
0.25	57	96	269	93	166	971	273	273	273
0.125	151	299	804	189	385	1678	563	561	563
`(Vm, states, Iion) = (p3, GP, p3)`									
Δx[mm]		P8 [ms]			N_{nl}			N_{lin}	
0.5	38.88	42.26	42.47	1 930	4 291	43 113	5 790	12 873	129 339
0.25	38.93	43.26	43.52	1 546	4 388	44 119	5 332	13 164	132 357
0.125	39.80	44.66	44.96	1 569	4 526	45 549	7 844	13 578	136 647
Δx[mm]		wall [min]			CPU [min]			RSS [MiB]	
0.5	40	75	197	64	128	723	211	211	211
0.25	127	296	702	146	350	1240	300	297	300
0.125	456	1069	2618	486	1149	3438	656	665	657
`(Vm, states, Iion) = (p3, CCp1, p1)`									
Δx[mm]		P8 [ms]			N_{nl}			N_{lin}	
0.5	75.42	57.19	57.29	4 320	7 956	79 678	12 960	23 868	239 034
0.25	51.56	56.76	56.99	2 981	5 741	57 626	10 672	17 223	172 878
0.125	43.26	48.42	48.66	2 144	4 904	49 263	10 719	14 712	147 789
Δx[mm]		wall [min]			CPU [min]			RSS [MiB]	
0.5	24	38	135	77	136	1103	205	205	205
0.25	59	101	298	95	171	1000	290	290	291
0.125	173	367	988	221	472	2089	616	619	619
`(Vm, states, Iion) = (p3, CCp3, p3)`									
Δx[mm]		P8 [ms]			N_{nl}			N_{lin}	
0.5	61.74	FAIL	FAIL	2 814	2 004	20 002	8 442	6 012	60 006
0.25	54.04	FAIL	FAIL	2 486	2 003	20 002	7 708	6 009	60 006
0.125	48.77	FAIL	FAIL	1 972	2 003	20 002	9 858	6 009	60 006
Δx[mm]		wall [min]			CPU [min]			RSS [MiB]	
0.5	20	15	52	54	40	296	220	219	219
0.25	67	59	188	97	84	433	334	334	334
0.125	208	230	681	253	275	1050	791	791	791

14.1.5 Unstructured triangular mesh, front-aware hybrid

Table 10: Per-case performance — unstructured triangular mesh, front-aware hybrid, all `Picard`. For each triplet `(Vm, states, Iion)` the rows list the available Δx [mm] and the columns the three Δt, grouped three metrics at a time. FAIL = no ignition (the run is aborted at $t = 0.02$ s by the failure detector); a P8 value implies full activation of the domain; `--` = combination not run. N_{nl} and N_{lin} are the totals of nonlinear and linear iterations (deterministic); wall and CPU in minutes; RSS in MiB.

Δt[s]	10^{-4}	10^{-5}	10^{-6}	10^{-4}	10^{-5}	10^{-6}	10^{-4}	10^{-5}	10^{-6}
`(Vm, states, Iion) = (NO, na, NO)`									
Δx[mm]		P8 [ms]			N_{nl}			N_{lin}	
0.5	55.81	59.70	59.88	3 000	6 060	60 760	12 000	18 179	121 520
0.25	45.18	50.28	50.51	2 231	5 095	51 168	15 615	16 180	102 336
0.125	40.93	46.00	46.30	1 662	4 657	46 858	18 281	23 284	140 574
Δx[mm]		wall [min]			CPU [min]			RSS [MiB]	
0.5	14	26	61	14	26	61	183	184	183
0.25	45	89	231	72	151	855	197	197	198
0.125	135	321	782	171	421	1797	247	247	247
`(Vm, states, Iion) = (NO, CCp1, p1)`									
Δx[mm]		P8 [ms]			N_{nl}			N_{lin}	
0.5	55.81	59.70	59.88	3 000	6 060	60 760	12 000	18 179	121 520
0.25	45.18	50.28	50.51	2 231	5 095	51 168	15 615	16 180	102 336
0.125	40.93	46.00	46.30	1 662	4 657	46 858	18 281	23 284	140 574
Δx[mm]		wall [min]			CPU [min]			RSS [MiB]	
0.5	15	26	66	15	26	66	185	186	185
0.25	46	90	241	73	152	864	204	204	206
0.125	138	330	878	174	429	1880	273	273	272
`(Vm, states, Iion) = (p3, CCp1, p1)`									
Δx[mm]		P8 [ms]			N_{nl}			N_{lin}	
0.5	75.42	57.19	57.29	4 320	7 956	79 678	12 960	23 868	239 034
0.25	51.56	56.76	56.99	2 981	5 741	57 626	10 672	17 223	172 878
0.125	43.26	48.42	48.66	2 144	4 904	49 263	10 719	14 712	147 789
Δx[mm]		wall [min]			CPU [min]			RSS [MiB]	
0.5	23	37	132	75	134	1097	205	205	205
0.25	57	98	299	93	168	999	289	289	291
0.125	162	314	978	205	413	2070	618	619	620
`(Vm, states, Iion) = (p3, CCp3, p3)`									
Δx[mm]		P8 [ms]			N_{nl}			N_{lin}	
0.5	38.88	42.27	42.55	1 930	4 293	43 188	5 790	12 879	129 564
0.25	38.93	43.30	43.77	1 543	4 391	44 368	5 321	13 173	133 104
0.125	39.80	44.71	45.30	1 570	4 531	45 883	7 850	13 593	137 649
0.1	39.86	44.81	45.40	1 575	4 535	45 925	9 449	13 605	137 775
0.05	39.62	44.61	–	1 586	4 513	–	15 856	18 052	–
Δx[mm]		wall [min]			CPU [min]			RSS [MiB]	
0.5	15	29	108	38	81	631	219	219	219
0.25	41	103	355	59	156	897	331	331	331
0.125	154	388	1409	189	489	2376	781	793	793
0.1	236	578	2135	271	675	3122	1126	1128	1126
0.05	896	2216	–	955	2341	–	3925	3883	–

Discussion of the 2D results (`Picard`). The whole sweep uses `Picard`; the `diagonalIion` and `JFNK` results in 2D are still pending. Relative to the previous version of this report, the sweep was extended to $\Delta x \in \{0.1, 0.05\}$ mm in the configurations that reach the converged regime, which for the first time makes it possible to fix a reference value and measure the error of each configuration against it.

Converged reference value. On the hexahedral mesh, the four paths that reach $\Delta x = 0.05\,\text{mm}$ with $\Delta t = 10^{-6}$ agree on

$$t_{\text{P8}}^{\star} \simeq 45.5\,\text{ms}, \tag{41}$$

with a spread of only 0.10 ms (0.2%): 45.60 ms (`(NO,na,NO)`), 45.50 ms (`(NO,GP,p1)`), 45.51 ms (`(NO,CCp1,p1)` hybrid) and 45.52 ms (`(p3,CCp3,p3)` hybrid). That four discretisations with different spatial orders, ionic-source treatments and state-integration modes converge to the same number is the strongest verification of the sweep: the solver is consistent, and the differences between configurations on coarse meshes are discretisation error, not model biases. All the error percentages that follow are measured against (41).

1. **High order in the ionic source speeds up and sharpens the front.** Raising `Iion` from `NO` to `p3` monotonically lowers the activation time: on hexa at $\Delta t = 10^{-6}$, $\Delta x = 0.25\,\text{mm}$, P8 goes from 67.4 ms (`(NO,na,NO)`) to 44.4 ms (`(p3,GP,p3)`). It is the same high-order-in-the-source effect as in the previous explicit and implicit reports, now resolved with `gaussPointODE`.

2. **The two families converge from opposite sides, and at very different speeds.** With $\Delta t = 10^{-5}$ s fixed and refining Δx on hexa:

Δx [mm]	0.5	0.25	0.125	0.1	0.05
`(NO,na,NO)`	157.15	67.30	50.05	47.86	45.28
`(NO,GP,p1)`	94.34	58.85	48.04	46.79	45.18
`(p3,CCp3,p3)` hybrid	36.30	44.22	45.43	45.48	45.04

   The low-order-source paths approach (41) *from above* and very slowly: `(NO,na,NO)` is still 10% high at 0.125 mm and needs 0.05 mm to drop below 1%. The high-order source path approaches *from below* and is already within 0.2% at 0.125 mm. In terms of cells: the high-order path reaches the target accuracy with 16× fewer cells.

3. **$\Delta t = 10^{-5}$ s is already enough; 10^{-6} s is wasted money.** Across all runs that activate, $|P8(10^{-6}) - P8(10^{-5})| \leq 0.5\,\text{ms}$ ($\lesssim 1\%$), whereas the cost is multiplied by between 4 and 10: on hexa, `(p3,CCp3,p3)` hybrid at 0.125 mm, the wall time goes from 205 to 845 min and N_{nl} from 4 607 to 46 527, to move P8 from 45.43 to 45.90 ms — that is, *away* from the converged value. By contrast $\Delta t = 10^{-4}$ s is **not** enough: it underestimates P8 by $\sim 4.6\,\text{ms}$ (10%) on the high-order path.

4. **Beware of error cancellation at coarse Δt.** The combination `(NO,na,NO)`, $\Delta x = 0.125\,\text{mm}$, $\Delta t = 10^{-4}$ s gives 45.03 ms, apparently a 1% error. It is a coincidence: refining Δt on the same mesh moves the result to 50.05 and 50.26 ms (10%). The temporal error (which underestimates P8) cancels the spatial error (which overestimates it). **An operating point is only valid if it is stable in Δt**; checking the proximity to the converged value is not enough.

5. **Pure `cellCentredReconstruct` fails when Δt is refined.** On hexa, the `CCp1`/`CCp3` configurations without hybrid activate at $\Delta t = 10^{-4}$ s but **do not ignite** (FAIL, $N_a = 0$, abort by ignition timeout at $t = 0.02\,\text{s}$) at $\Delta t = 10^{-5}$ and 10^{-6} s, and it is now confirmed that the failure persists also at $\Delta x = 0.1$ and 0.05 mm: **it is not a problem of spatial resolution**. On the triangular mesh the same happens with `(p3,CCp3,p3)`. The behaviour is governed entirely by Δt and is independent of Δx over a 10× range; its cause is under analysis and is not attributed here.

6. **The front-aware hybrid rescues `CC` at the cost of partial `GP`.** With `frontAwareHybrid true`, the same `CCp`k configurations activate at *all* the Δt, with activation times practically equal to those of `gaussPointODE` but much cheaper. At $\Delta x = 0.125\,\text{mm}$, $\Delta t = 10^{-5}$ s: `(p3,CCp3,p3)` hybrid gives 45.43 ms in 205 min, against 45.40 ms in 932 min for `(p3,GP,p3)` without hybrid — same answer, 4.5× cheaper, with the same N_{nl} ($\approx 4\,600$). The reduction comes exactly from what was warned above: the hybrid integrates the N_q ODEs per cell only in the front band and one per cell everywhere else, so that at equal numbers of nonlinear iterations the ODE work per

evaluation is much smaller. The hybrid therefore combines the *correction* of `GP` with a good part of the *saving* of `CC`.

7. **Cost scaling.** N_{nl} and N_{lin} grow ∼linearly when refining Δt (more steps). The wall time grows approximately as Δx^{-2} (number of cells in 2D): in `(p3,CCp3,p3)` hybrid at $\Delta t = 10^{-5}$, going from 0.125 to 0.05 mm (6.25× cells) takes the wall time from 205 to 1 123 min (5.5×) and the RSS from 530 to 2 367 MiB (4.5×). The most expensive case of the sweep is `(p3,GP,p3)` without hybrid on hexa at $\Delta t = 10^{-6}$, $\Delta x = 0.125$ mm: 2 228 min (∼ 37 h). The RSS grows with the LRE stencil and with $1/\Delta x^2$, and is essentially independent of Δt.

8. **Wall vs. CPU: why the deterministic quantities are reported.** At $\Delta x \leq 0.125$ mm the CPU time exceeds the wall time (multi-threaded execution under contention; e.g. `(NO,na,NO)` hexa $\Delta t = 10^{-6}$: wall 339 min, CPU 1098 min), whereas on coarse meshes they coincide. That divergence is precisely the machine-dependent noise that motivates using N_{nl} and N_{lin} as a reproducible cost reference.

9. **Triangular mesh.** The qualitative pattern is preserved (failure of pure `CC`, rescue by the hybrid, high order in I_{ion} speeding things up). Three observations of its own: (i) on the triangular mesh, the configurations with `Iion=p1` coincide *exactly* with the baseline `(NO,na,NO)` (same P8, same N_{nl}, same N_{lin}) — the $p1$ reconstruction of I_{ion} contributes nothing on this unstructured mesh, and only `p3` separates; (ii) the converged triangular value, $\simeq 44.6$ ms for `(p3,CCp3,p3)` hybrid at 0.05 mm, lies ∼ 2% below the hexahedral one (41) — a genuine mesh difference, not a method one; (iii) the larger number of cells per Δx (5 954 vs 2 560 at 0.25 mm) proportionally increases every quantity, so that at equal Δx the triangular case costs between 1.9 and 2.4× more wall time.

Recommended configuration. Combining accuracy and cost, and taking (41) as the reference, the sweep identifies one clearly dominant operating point and two alternatives depending on the budget:

Use	Triplet / hybrid	Δx [mm]	Δt [s]	P8 [ms]	error	wall [min]
Recommended	`(p3,CCp3,p3)` + hybrid	0.125	10^{-5}	45.43	0.2%	**205**
Quick exploration	`(p3,CCp3,p3)` + hybrid	0.25	10^{-5}	44.22	2.8%	**60**
Reference	`(p3,CCp3,p3)` + hybrid	0.05	10^{-6}	45.52	0.05%	4 211
Alternatives at the same error level, for comparison:						
Low order	`(NO,na,NO)`	0.05	10^{-5}	45.28	0.5%	739
High order, no hybrid	`(p3,GP,p3)`	0.125	10^{-5}	45.40	0.2%	932

Why that combination. The three ingredients are all necessary and each one solves a different problem:

- `Iion=p3` with `CCp3` states gives the fast spatial convergence (point 2): it is what allows stopping at 0.125 mm instead of 0.05 mm.
- `frontAwareHybrid true` is what makes `CCp3` *ignite* (point 5) and, at the same time, avoids paying the N_q ODEs per cell over the whole domain (point 6).
- $\Delta t = 10^{-5}$ s is the coarsest step that gives temporal stability without overspending (point 3).

Gain. Against the fully resolved reference, the recommended configuration is 20× cheaper (205 against 4 211 min) for 0.2% error. Against the low-order alternative at equal accuracy it is 3.6× cheaper (205 against 739 min) and uses 16× fewer cells; against the same physics without hybrid, 4.5× cheaper. If only a qualitative sweep is needed, dropping to 0.25 mm costs 60 min (70× less than the reference) in exchange for an error of 2.8%, which is still smaller than the 10% of the classical baseline at 0.125 mm.

What to avoid. (i) `CCp`k without hybrid at any $\Delta t \leq 10^{-5}$ s: it does not ignite, and the failure is not fixed by refining the mesh. (ii) $\Delta t = 10^{-4}$ s as a production operating point: it can give an

apparently good number by error cancellation (point 4) without being reproducible under refinement. (iii) $\Delta t = 10^{-6}$ s: it multiplies the cost by 4–10 without improving the answer. (iv) `(p3,GP,p3)` without hybrid: it gives the same answer as the recommended one but 4.5× more expensive.

14.2 3D Niederer-style benchmark

The 3D sweep follows Niederer's published slab geometry ($20 \times 3 \times 7$ mm, $40 \times 6 \times 14$ cells at $\Delta x = 0.5$ mm). At the time of writing, only the `diagonalIion` and `Picard` sweeps include 3D runs, all at $\Delta x = 0.5$ mm. The 3D `JFNK` sweep has not been run yet.

Table 11: 3D P8 activation time [ms] at $\Delta x = 0.5$ mm, $\Delta t = 10^{-4}$ s. The columns are the available nonlinear methods.

Vm	Iion	`Picard`	`diagonalIion`
NO	NO	136.67	136.61
NO	p1	33.50	33.39
NO	p3	30.59	30.51
p3	NO	149.13	149.06
p3	p1	32.44	32.36
p3	p3	29.63	NC

Table 12: 3D solver wall-clock time [min] at $\Delta x = 0.5$ mm, $\Delta t = 10^{-4}$ s.

Vm	Iion	`Picard`	`diagonalIion`
NO	NO	80	82
NO	p1	168	229
NO	p3	1489	2159
p3	NO	163	143
p3	p1	167	269
p3	p3	1487	NC

Table 13: 3D peak resident set size [MiB] at $\Delta x = 0.5$ mm, $\Delta t = 10^{-4}$ s.

Vm	Iion	`Picard`	`diagonalIion`
NO	NO	156	157
NO	p1	211	211
NO	p3	787	849
p3	NO	1123	1121
p3	p1	1164	1162
p3	p3	1565	NC

Discussion of the 3D results. On the coarse 3D slab ($\Delta x = 0.5$ mm), enabling LRE only in the ionic source changes the activation time from ≈ 136.7 ms (`NO/NO`) to ≈ 33.5 ms (`NO/p1`) and ≈ 30.6 ms (`NO/p3`) — the same qualitative high-order-in-the-source effect already seen in the 2D sweep and in the explicit/implicit Niederer reports. The fully high-order case (`p3/p3`) reaches 29.63 ms with Picard (the diagonalIion counterpart has not been completed yet). The two nonlinear methods again agree to better than 0.1 ms for every entry where both are available; Picard is generally faster, with the gap most noticeable in the heaviest case (`NO/p3`: 1489 min for Picard vs. 2159 min for

diagonalIion). The peak RSS scales with the LRE stencil size and is essentially independent of the method. The data set is too sparse in Δx to draw quantitative conclusions about mesh convergence in 3D; the immediate gap to fill is the missing `p3/p3` diagonalIion entry and a 3D JFNK sweep on the same mesh.

A Dimension table

Field	OpenFOAM `dimensionSet`	SI
`Vm`	$[1, 2, -3, 0, 0, -1, 0]$	volt
`Iion`	`dimVoltage/dimTime`	$\mathrm{V\,s^{-1}}$
`externalStimulusCurrent`	`dimCurrent/dimVolume`	$\mathrm{A\,m^{-3}}$
`conductivity`	`dimCurrent^2 dimTime^3 / (dimMass dimVolume)`	$\mathrm{S\,m^{-1}}$
`chi`	`dimless/dimLength`	$\mathrm{m^{-1}}$
`Cm`	`dimCurrent dimTime / (dimVoltage dimArea)`	$\mathrm{F\,m^{-2}}$

B File map

Path (relative to `cardiacFoam-pc/`)	Role
`applications/solvers/highOrderElectroActivationFoamImplicitPETScDistributed/` `highOrderElectroActivationFoamImplicitPETScDistributed.C`	Main transient driver (time loop, nonlinear solvers, Eigen/PETSc linear backends, distributed `MPIAIJ` operators, LRE reconstruction, activation tracking).
`applications/solvers/highOrderElectroActivationFoamImplicitPETSc/`	Serial `...PETSc`, and `...PETScParallel` the intermediate step (parallel stencils, serial algebra). Kept as references (Section D).
`applications/solvers/highOrderManufacturedFDAImplicitPETScDistributed/`	MMS verification sibling sharing the parallel design; it carries the coupled-face stabilisation exchange and the `CF_DUMP_K` / `CF_DUMP_STENCILS` diagnostics (Section D.4).
`src/highOrderAdapter/highOrderInterp.H`	Adapter over solids4foam's `movingLeastSquares`: stencils with global columns, halo exchange, reconstruction coefficients.
`src/ionicModels/`	Ionic models (TNNP and others): embedded CellML kernel, state update, current evaluation and local ODE integration.
`applications/solvers/.../createFields.H`	Field initialisation and dictionary reading.
`tutorials_electro/.../run_convergence.py`	Driver script.
`constant/cardiacProperties`	χ, C_m, $\mathbf{D}$, ionic model.
`constant/timeIntegrationProperties`	ODE solver, tolerances, sub-step cap.
`constant/spatialIntegrationProperties`	Time scheme, HO flags, JFNK / linear / line-search options.
`constant/stimulusProtocol`	Stimulus regions and timing.

C Linear-algebra back-ends and sibling solver

This solver *integrates both linear-algebra back-ends* in a single executable, selectable at run time with `linearSolverBackend` (assembled branches) and `jfnkLinearSolverBackend` (JFNK Krylov). Matrix assembly and storage always use `Eigen::SparseMatrix`; the back-end only determines how the system is solved.

Aspect	`...Backend = Eigen`	`...Backend = PETSc`
Matrix container	`Eigen::SparseMatrix`	idem (copied to `Mat`)
Sparse direct solver	`Eigen::SparseLU`	`KSPPREONLY+PCLU`
Iterative solver (Picard/diag)	`Eigen::BiCGSTAB+ILUT`	`KSP+PC` (`gmres+ilu`, `cg+gamg`, ...)
JFNK inner Krylov	In-house GMRES	`KSPGMRES` over `MatShell`
JFNK preconditioner	`jfnkPreconditioner`: `none` or `diffusion` (ILU of $\mathbf{A}$) — available in both	

The convergence parameters (`nonlinearTolerance`, `jfnkEpsilon`, `jfnkLineSearch*`, `jfnkInitGuess*`, `jfnkClampODEInput`, `nonlinearAcceptUnconverged`, `ODE_*`) have the same meaning regardless of the back-end, so that the same `run_convergence.py` configuration produces an algorithmically equivalent run (apart from the performance differences of the linear solve).

Sibling solver `highOrderElectroActivationFoamImplicit`. There is also a simpler version, with an *Eigen-only* back-end and without the optimisations of Section E. Its main algorithmic difference is that it runs a *fixed* number of nonlinear iterations (`nonlinearIterations`, 1 by default) *without* a convergence test: with the default value it is a semi-implicit scheme with *one* ODE evaluation and *one* linear solve per time step. This makes it much faster at the cost of not controlling the nonlinear residual; at the small time steps of the benchmark the difference in the results is marginal (see Sections E and F).

D Distributed-memory parallelisation

This variant, `highOrderElectroActivationFoamImplicitPETScDistributed`, leaves every algorithm of the previous sections unchanged and replaces the linear algebra with a genuinely distributed one. The distinction matters, because there are three solvers in the family and only the last one is parallel in the algebraic sense:

Variant	Linear algebra
`...ImplicitPETSc`	Serial mesh, serial LRE, Eigen assembly, PETSc as a solver on `PETSC_COMM_SELF`.
`...ImplicitPETScParallel`	Parallel high-order library: the mesh is decomposed and the stencils cross ranks, but the assembly still produces one Eigen matrix per rank and hands the finished CSR to PETSc on `PETSC_COMM_SELF`. Serial by construction.
`...ImplicitPETScDistributed`	Matrices are `MPIAIJ` on `PETSC_COMM_WORLD` and every matrix–vector product goes through `MatMult`.

The three are kept side by side deliberately: each is the reference against which the next one is checked, rather than being compared from memory.

D.1 Matrix layout: local rows, global columns

Each rank owns the rows of its own cells and nothing else. Columns, in contrast, are *global* cell indices, taken from the `globalIndex` that the high-order library builds together with its stencils. Writing n_{loc} for this rank's cell count and n_{glob} for the total, every assembled operator has shape $n_{\text{loc}} \times n_{\text{glob}}$, and the row offset of this rank is

$$\texttt{gRowStart} \;=\; \texttt{globalIndex::localStart()}.$$

That is exactly PETSc's `MPIAIJ` layout, so the translation from the Eigen container to the distributed matrix (`buildDistributedMat`) only has to offset the row index and split the preallocation into the diagonal block (columns this rank owns) and the off-diagonal one. Getting that split right is what stops assembly from degenerating into repeated reallocation.

In serial `gRowStart` $= 0$ and $n_{\text{glob}} = n_{\text{loc}}$, so the same code path produces exactly the sequential matrix it replaces.

A bug family worth naming. That last property is convenient and dangerous in equal measure: *a local index used where a global one is expected is invisible in serial.* Five instances were found in this family of solvers, all of them silent at one rank:

Site	Symptom at > 1 rank
Diagonal of the mass matrix	Extra nonzeros in $\mathbf{A}$, exactly the per-rank row count.
`addDiagonalToMatrix` column	As above.
`updateValues` size check	Accepts a vector of the wrong length.
`updateValues` row offset	Values written into the wrong rows.
Orthogonal assembly of the manufactured sibling	Local columns throughout, so the operator could not be assembled correctly at all.

Section D.4 explains why the usual sanity checks do not see any of these.

D.2 What has to cross a processor boundary

Three kinds of data cross a partition, and each needs a different mechanism.

(i) Reconstruction stencils — nothing to do. The high-order library builds its stencils geometrically over a halo whose depth is controlled by `haloDepthScale`, and emits *global* column indices. The flux term at a processor face therefore assembles exactly as at an internal face: the stencil already reaches across the cut.

(ii) Cell-centred data — `syncTools`. The conductivity of the cell on the far side of a coupled face, and its global index, are obtained with `syncTools::swapBoundaryCellList`, indexed by boundary-face position. Two tempting alternatives are both wrong:

- a processor patch *field* holds the interpolated value *at the face*, not the value of the neighbouring *cell*;
- `mesh.C()` is a `slicedVolVectorField` whose coupled values are only valid after an evaluation that the assembly does not perform.

For the same reason the owner-to-neighbour vector $\mathbf{d}_{PN}$ is taken from `patch.delta()`, which for a `processorFvPatch` returns $\mathbf{C}_N - \mathbf{C}_P$ built from both sides' geometry, and never from `C[nei] - C[own]`.

(iii) Variable-length matrix rows — `PstreamBuffers`. The stabilisation jump is the one term that needs the neighbouring cell's whole reconstruction row: up to ~ 60 (global column, coefficient) pairs for a $p3$ stencil in 3-D. No field swap can carry that.

The mechanism, implemented in the manufactured sibling `highOrderManufacturedFDAImplicitPETScDistributed` is to let the rank that *owns* the cell build the row itself and ship the pairs, evaluated at the face centre — a point both ranks have. Nothing else is exchanged: no cell centres, no stencils, no coefficient tables. Writing the jump from the point of view of the local cell,

$$\frac{a}{V_c}\Big(T_{\mathrm{far}}(\mathbf{x}_f) - T_c(\mathbf{x}_f)\Big),$$

where T_c is the Taylor reconstruction of cell c, also removes the owner/neighbour convention: the $+$ that the internal-face loop writes into the owner row and the $-$ it writes into the neighbour row are the same expression seen from either cell, and the face coefficient a depends on the orientation only through $|\mathbf{d}_{PN}\cdot\mathbf{n}|$ and $\mathbf{n}\cdot\mathbf{D}\cdot\mathbf{n}$. Both ranks therefore compute the same a without agreeing on anything.

Status in this solver. The coupled branch of the orthogonal assembly adds the flux term and *not* the Rhie–Chow jump. This is inert at the default $\alpha = 0$ that the benchmarks use, and it is documented in the source rather than silently skipped; porting the exchange described above is the outstanding item.

D.3 Global reductions

Everything that used to be a loop over all cells and is now a loop over this rank's cells needs a reduction: the error norms of Section 14 (`gSum` over volumes and squared errors), the nonlinear residuals and their stopping tests, and the activation-time sampling.

The one that is easy to miss is the *Armijo backtracking line search*, condition (36). Its accepted step must be a single number for the whole domain; without the reductions inside the search, each rank would accept a different λ and the Newton iterate would stop being a function of the mesh alone. That the parallel run reproduces the serial `lineSearchIters` *exactly* is what validates this.

D.4 Verifying a parallel operator

Two checks are cheap and are *not* sufficient on their own:

- **Row sums.** With no-flux boundaries a constant field has zero gradient, so every row of $\mathbf{K}$ must sum to zero. This catches an empty or malformed operator — it caught a `CompactListList::operator[]` returning a view by value, which compiled but produced an empty stencil — but it cannot see an *omitted* face, because omitting a face contributes nothing to either of its rows and the sums still vanish.
- **Operator fingerprint** (global nnz, $\sum a_{ij}$, $\sum |a_{ij}|$, $\max_i |\sum_j a_{ij}|$). Since the stencils are geometric, the global nnz must match the serial value exactly. But a value written into the *wrong column* changes neither nnz, nor any sum, nor any row sum.

The check that does decide is an entry-by-entry comparison. With `CF_DUMP_K` set in the environment, the manufactured sibling writes $\mathbf{K}$ as (global row, global column, value), one file per rank; mapping the parallel indices back through `processor`N`/constant/polyMesh/cellProcAddressing` gives a direct diff against the serial assembly. `CF_DUMP_STENCILS` does the same one level lower, for stencil *membership* and cell centres, which is what distinguishes a differing coefficient from a differing stencil.

Comparing iteration by iteration. Use `petscLinearPcType jacobi`. PETSc wraps `ilu` and `lu` in block-Jacobi in parallel, so those preconditioners are partition-dependent *by construction*: with `ilu` a converged manufactured case shows a 4.5×10^{-6} deviation between one and two ranks that is entirely algebraic, and with `jacobi` the same case agrees to all printed digits. Mistaking that for a discretisation error costs a lot of time.

D.5 Partition dependence of the MLS stencils

Note (2026-10). In the MMS solver it was later found that much of this discrepancy came from the case and not from the library: the `Nn` key is the number of cells *added* on top of the minimum for the degree, and the sweeps wrote it as if it were the total, so the stencils were 50 % larger than intended and the halo did not cover them. With the correct size, in 2-D, M11 measures 0 differing stencils between np=1 and np=2 and M15 drops to 7×10^{-15}. The 3-D measurements below predate that fix and were not re-run.

One genuine parallel inconsistency remains, and it lives in the stencil selection of the high-order library, not in any solver.

Mechanism. A stencil is meant to be *the N cells whose centres are nearest the face centre*. The library reaches that in two steps: it first builds a pool of candidates by growing complete layers of `cellCells` until the pool holds $\lceil 1.5N \rceil$ cells, and then keeps the nearest N of the pool by Euclidean distance. The second step is geometric; **the first is topological**. On a tetrahedral mesh "within k hops" and "within radius r" are not the same set, and with only $0.5N$ of slack the last positions of the stencil are decided by the connectivity rather than by the geometry.

Under decomposition the layer growth stops at the cut, and the far side is supplied instead by a query that returns every cell of the neighbouring rank inside a bounding box — a *geometric* set, and a generous one. Near a partition the selection therefore mixes a topological candidate set on one side with a geometric one on the other. This is why the deviation does not respond to `haloDepthScale`: the box was never the binding constraint.

Measurements (3-D unstructured tetrahedra, 4971 cells, one rank against two, $\alpha = 0$ so that no stabilisation is involved):

Observable	Value
Face stencils that differ	536 of 10676
Their size	identical (80): members are *substituted*, not lost
Position of the swapped members	98 % in the last 10 % of the distance-ordered stencil
Rows of $\mathbf{K}$ that differ	552 of 4971, up to 4.17 against $\max \lvert K \rvert = 256$
Cut faces whose two ranks disagree with *each other*	9
Cell centres, serial vs decomposed	up to 3.3×10^{-16} (the centroid sum runs over a renumbered face list)

The nine cut faces are a defect on their own terms, with no serial comparison needed: the two rows of one face are then built from different flux stencils, so the flux leaving a cell is not the flux entering its neighbour and the scheme is not conservative there.

How much it matters. Comparing the V_m *fields* cell by cell rather than the deviation of a norm, at $\alpha = 0.1$ with `jacobi`:

$$\frac{\left\| V_\mathrm{m}^{np=1} - V_\mathrm{m}^{np=2} \right\|}{\| V_\mathrm{m} - V_\mathrm{m}^{\mathrm{exact}} \|} = \frac{1.51 \times 10^{-5}}{6.84 \times 10^{-4}} = 2.2\,\%, \qquad \frac{\| \Delta V_\mathrm{m} \|}{\text{amplitude}} = 7.5 \times 10^{-6}.$$

Over a refinement sweep $N = 8, 10, 12, 14$ the deviation stays at the 10^{-4}–10^{-3} level while the error itself falls from 1.19×10^{-3} to 2.28×10^{-4}: the perturbation shrinks at the same rate as the error. Consequently the fitted convergence order is untouched,

$$p_{V_\mathrm{m},L_2}^{np=1} = 2.90722821, \qquad p_{V_\mathrm{m},L_2}^{np=2} = 2.90722635,$$

a difference of 1.9×10^{-6} against an acceptance tolerance of 0.05. That is what two *consistent* schemes of the same formal order predict: their difference is a same-order perturbation, not a systematic error.

How it would be fixed.

1. Make the candidate pool geometric on both sides — query cell centres by radius (`indexedOctree`) instead of growing `cellCells` layers, with the radius taken from the layer estimate as an upper bound. The stencil then becomes a function of (face centre, cell centres, N) alone, invariant under partitioning.
2. Stop the tie clipping: the parallel branch filters candidates by radius *before* the equal-distance expansion runs, so a tie group straddling that radius is cut in half (17 members in serial against 16 at two ranks, on a structured 2-D case). Inflating the radius by $(1 + \texttt{relTol})$ and routing the serial early return through the same selection code closes it.
3. Assert completeness instead of failing silently: check that the N-th distance lies strictly inside the queried radius.
4. One face, one stencil: have the lower-ranked side of a cut face own the stencil and send it, which restores conservation across the cut.

`relTol` (10^{-8}), `overSampleFactor` (1.5) and `searchExpansionFactor` (1.3) are hard-coded default arguments, not dictionary entries, so none of this can be explored from a case: it is a library change.

A caveat before touching it. The serial run is the truncated one — its candidate pool is purely topological everywhere, not only near a cut — so fixing the selection changes serial results too, and with them every convergence table produced so far. That is a decision to take deliberately, not a side effect to absorb.

D.6 Verification cases: what was run and where it lives

Every case below sits under `tutorials_electro/electroTestParallelVerification/`. The driver is `tutorials_electro/electroDistValidation/`; the directories here are copies of it with one setting changed each.

Directory	Test	What it establishes
`electroA3_np1`, `_np2`, `_np4`	A3, physical acceptance	activation time at P8 and conduction velocity, full run to activation
`electroA4_np2`, `_np4`	A4, decomposition invariance	`simple` instead of `scotch`: a different cut geometry
`electroTri_np1`, `_np2`, `_np4`	unstructured mesh	the gmsh route, 1500 cells
`matrix_el_2D_np1`, `_np2`, `matrix_el_3D_*`	method matrix	Picard / JFNK / diagonalIion, hexa + triangular, 2-D and 3-D
`jac_2D_np1`, `jac_2D_np2`	preconditioner isolation	`jacobi`, the only rank-independent preconditioner
`perf_{ilu,jac,gamg,hypre}_np{1,2}`	linear-solver performance	$\Delta x = 0.5\,\text{mm}$, 640 cells
`perf025_{ilu,jac,gamg,hypre}_np{1,2}`	idem, refined	$\Delta x = 0.25\,\text{mm}$, 2560 cells
`pre_{0p0,0p1}_np{1,2}`	stabilisation, before	built with the `electroPreStab` binary
`post_{0p0,0p1}_np{1,2}`	stabilisation, after	built with the ported binary
`stab_{0p0,0p1}_np{1,2}`	first baseline attempt	*discarded*: rebuilds landed mid-sweep

A3, physical acceptance (`electroA3_*`). Hexa, $\Delta x = 0.5\,\mathrm{mm}$, $\Delta t = 10^{-4}$, triad $(p3, \mathrm{CCp3}, p3)$, run until the wavefront reaches P8. Sequential runs on an idle machine, so the wall times are valid.

Ranks	Status	T_{P8} [ms]	dev.	CV [m/s]	Wall [s]	Speed-up
1	OK	52.0795930112	—	0.38388	310.3	1.00×
2	OK	52.0795930123	2.11×10^{-11}	0.38388	177.0	1.75×
4	OK	52.0795930133	4.03×10^{-11}	0.38388	102.9	3.01×

`OK` is the most important entry in each row: it is the binary conduction-block detector, and it says the wavefront crosses every partition cut. The 0.5 % tolerance proposed for $|\Delta T_{P8}|/T_{P8}$ was *wrong by eight orders of magnitude* — the observed spread is 4×10^{-11}. Picard converges to the same fixed point deterministically, so a loose stopping residual introduces no drift.

A4, decomposition invariance (`electroA4_*`). The same run cut with `simple` ($n = (2\,1\,1)$ and $(2\,2\,1)$) rather than `scotch`: `OK` in both, T_{P8} within 9.22×10^{-11} of serial, CV identical to five decimals. Two unrelated cut geometries agreeing is far stronger evidence than several rank counts under one method.

Unstructured mesh (`electroTri_*`). 1500 cells through the gmsh route: `OK` at 1, 2 and 4 ranks, T_{P8} within 6.55×10^{-11}, speed-up 1.65× and 2.84×.

Linear-solver performance (`perf_*`, `perf025_*`). Krylov iterations per nonlinear iteration vary by more than a factor three between preconditioners while the wall times agree to within 8 %, at both mesh sizes. *The linear solve is not the bottleneck* — the ionic ODEs are — so changing preconditioner buys nothing here. Refining $0.5 \to 0.25\,\mathrm{mm}$ multiplies the cost by 4.3, i.e. essentially linearly in cell count, and the iteration counts stay flat (ilu 3.0, hypre 4.0, jacobi 8.0), so no crossover appears. `gamg` failed on the finer mesh and was not diagnosed.

Preconditioner isolation (`jac_2D_*`). `ilu` has no MPIAIJ implementation and is wrapped in block-Jacobi in parallel, which is partition-dependent by construction; `jacobi` is purely diagonal and therefore the same operator at every rank count. Switching to it collapsed the $np = 1$ vs $np = 2$ trajectory differences: five of six configurations became *identical*, and the row counts matched in all six (`triangular` with Picard went from 184 vs 275 rows to 184 vs 184).

Processor-face stabilisation (`pre_*` vs `post_*`). Deviation of the residual trace between $np = 1$ and $np = 2$ at $\alpha = 0.1$:

Mesh	Method	Before	After	$\alpha = 0$ control	Gain
hexa	Picard	3.23×10^{-9}	9.06×10^{-12}	8.53×10^{-12}	356×
hexa	JFNK	2.58×10^{-5}	2.59×10^{-5}	2.48×10^{-5}	1×
hexa	diagonalIion	1.27×10^{-8}	4.51×10^{-10}	2.28×10^{-10}	28×
triangular	Picard	2.45×10^{-7}	4.68×10^{-11}	9.55×10^{-12}	5240×
triangular	JFNK	4.44×10^{-6}	4.43×10^{-6}	4.64×10^{-6}	1×
triangular	diagonalIion	1.59×10^{-7}	6.17×10^{-9}	5.59×10^{-10}	26×

Serial output is bit-identical (SHA-256) in all twelve configurations, at both α values. The assembled paths collapse onto the $\alpha = 0$ control, which is the achievable floor. JFNK does not move, and that is expected: its deviation is dominated by its own preconditioner — the Eigen ILUT of the diagonal block, partition-dependent by construction — not by the assembled stabilisation. The same 2.48×10^{-5} appears with `ilu` and with `jacobi`, which is what identified it.

Two *distinct* binaries were built for this comparison, `electroPreStab` and the ported solver, because a first attempt (`stab_*`) had rebuilds landing while the sweep was still running. That

matters: had the "before" already contained the fix, the "after" would have been identical, and a null result is indistinguishable from a working one.

E Performance optimisations

This section documents four optimisations added to the solver. All of them are *additive* and are governed by dictionary keys whose default value reproduces **exactly** the previous behaviour: no existing run changes its result unless they are explicitly enabled. They also update some rows of the comparison table of the previous Section (the assembled back-end now reuses the factorisation, JFNK admits a preconditioner, and the ionic ODE can be integrated in parallel with OpenMP).

E.1 Matrix and factorisation reuse (Phase B)

The implicit operator $\mathbf{A} = \mathbf{M}/\Delta t - \theta\,\mathbf{L}$ depends only on the constant matrices $\mathbf{M}$, $\mathbf{L}$, on the parameter θ and on the step Δt. Since Δt is fixed except in the last clipped step, $\mathbf{A}$ is constant during almost the whole simulation. Previously, $\mathbf{A}$ was reassembled and **re-factorised** at every nonlinear iteration; now it is assembled once and the factorisation (PETSc KSP, or `Eigen::SparseLU/BiCGSTAB`) is kept for the whole run. An update policy distinguishes three cases per solve:

- **Rebuild**: first use or change of $\Delta t \Rightarrow$ full setup (allocation + symbolic + numeric factorisation).
- **ValuesOnly**: same sparsity pattern, new values (the diagonal changes at every iteration in `diagonalIion`) $\Rightarrow$ the values are refreshed reusing the pattern (`updateValues`).
- **Reuse**: matrix identical to the previous solve (Picard with a constant operator) $\Rightarrow$ the matrix is not touched, only solved.

The key `petscReusePreconditioner` (`false` by default) additionally allows, in `diagonalIion` mode, reusing the preconditioner across inner iterations when the diagonal change is small. In Picard this optimisation is purely computational: the same matrix is solved with the same factorisation, so the results are bit-for-bit identical (Section F.1).

E.2 Diffusion preconditioner for JFNK (A3)

The matrix-free JFNK Jacobian used to be solved without a preconditioner, so that each Krylov iteration requires a full residual evaluation (and therefore an ionic ODE integration). A *right* preconditioner $\mathbf{M}^{-1}$ is added with $\mathbf{M} = \mathbf{A}$, the linear (diffusive, SPD) part of the Jacobian. To avoid the catastrophic fill-in of an exact LU factorisation of the (wide-stencil) high-order operator, an **incomplete** LU **factorisation** (`Eigen::IncompleteLUT`, ILUT) is used, computed once and reused; each Krylov step then costs a single cheap *triangular solve*. It is very effective here because, with small Δt, $\mathbf{A} \approx \mathbf{M}/\Delta t$ is strongly diagonally dominant. The key `jfnkPreconditioner` takes the values `none` (default, legacy path) or `diffusion`. In the PETSc branch `KSPSetPCSide(PC_RIGHT)` is set so that the KSP monitors the true residual $\|b - Ax\|$; in the in-house GMRES branch (Eigen) the preconditioner is applied in the Arnoldi step and in the reconstruction of the correction. It does not change the converged solution, only the convergence path (fewer matvecs, i.e. fewer ODE integrations).

E.3 Parallel ionic ODE integration (C1)

The loop over the integration points of `TNNP::solveODE` is *embarrassingly parallel* (each point is an independent ODE). It is optionally parallelised with OpenMP through the key `parallelODE` (`false` by default) read from the ionic-model dictionary. Since the base class `ionicModel` shares a single `ODESolver` (with internal scratch for the Runge–Kutta stages), a per-thread solver *pool* is added (`ensureOdeSolverPool/odeSolver(tid)`); the diagnostic index `activeIntegrationPoint_` becomes per-thread and writing the protection log is serialised with `#pragma omp critical`. It was necessary to add `-fopenmp` to the compilation options of the `ionicModels` library. The number of threads is controlled with the environment variable `OMP_NUM_THREADS`; `run_convergence.py` exposes it through

the constants PARALLEL_ODE and ODE_NUM_THREADS. The results are numerically identical to the serial version (validated bit for bit).

E.4 AMG preconditioners (C3)

The KSP path already supported passing PETSc options at run time, so that the algebraic multigrid preconditioners (native `gamg`, `hypre` BoomerAMG) are directly reachable. The aliases `amg`→`gamg` and `boomeramg`→`hypre` are added to the `petscLinearPcType` mapping. For the SPD monodomain operator the theoretically advisable combination on large 3D meshes is `cg`+`gamg`; however, the experiments (Section F.5) show that for the time steps used here the system is mass-dominated (well conditioned) and AMG contributes nothing.

Summary of new keys. All with a default value equal to the previous behaviour.

Key	Default	Effect
`petscReusePreconditioner`	`false`	Reuses the preconditioner across iterations in `diagonalIion`.
`jfnkPreconditioner`	`none`	`diffusion`: preconditions the JFNK Jacobian with an ILUT of $\mathbf{A}$.
`parallelODE`	`false`	Integrates the ionic ODEs with OpenMP (ionic-model dict.).
`petscLinearPcType`	`ilu`	Now accepts `gamg`/`hypre` (and the aliases `amg`/`boomeramg`).

F Performance experiments

The above optimisations were measured on the 2D Niederer-style benchmark (23 726-cell mesh unless stated otherwise) with the TNNP model and the RKF45 integrator, Crank–Nicolson scheme and consistent mass in the high-order cases. Unless stated otherwise, the stimulus is the benchmark one (a 1.5 mm corner). The runs are short (few steps) and the measurements were taken with `/usr/bin/time -v`; where relevant, `loopWall` (time-loop time, from `solverPerformance.dat`) is reported to isolate the per-step cost from the one-off setup. The machine has 20 cores.

F.1 Phase B: identical and faster in Picard

The same runs of the convergence sweep were compared against a binary *prior* to the changes (Picard, Crank–Nicolson, consistent mass, RKF45, PETSc back-end `gmres`+`ilu`), taking the four cases present in both folders.

Δt / Δx	P8 [ms] (new=old)	nonlin. iter.	lin. iter.	wall new/old [s]	Δ
10^{-4} / 0.25 mm	62.9948077742	3510=3510	12748=12748	1488.7 / 1645.3	−9.5%
10^{-4} / 0.5 mm	153.3156018900	6293=6293	18879=18879	684.0 / 752.0	−9.0%
10^{-5} / 0.25 mm	67.2950937434	6854=6854	13708=13708	2531.9 / 2786.9	−9.2%
10^{-5} / 0.5 mm	157.1464148850	16136=16136	32272=32272	1472.6 / 1616.7	−8.9%

Conclusion. The P8 activation time agrees to ten decimals ($\Delta = 0$) and the number of nonlinear and linear iterations is identical, while the wall time drops by $\sim 9\%$. Since CPU usage was 99% (serial), the saving comes entirely from not re-factorising the constant matrix, not from parallelism.

F.2 A3: diffusion preconditioner in JFNK

Same case (corner, 10 steps, JFNK, PETSc back-end), varying `jfnkPreconditioner`.

`jfnkPreconditioner`	Krylov iter. (total)	wall [s]
`none`	90	98.6
`diffusion`	24	59.0

Conclusion. The preconditioner reduces the matvecs from 90 to 24 (3.75× fewer ODE integrations) and the wall time by 1.67× (98.6 → 59.0 s), with the same solution (Vm agreeing within the Krylov tolerance). Preconditioning is clearly faster than not doing it.

F.3 C1: ODE parallelisation (does not help in this problem)

A/B test with `OMP_NUM_THREADS` fixed at 8, toggling `parallelODE` (the difference isolates the effect of C1, since the internal PETSc/BLAS threading is the same in both). For the high-order and 3D cases `loopWall` is reported.

Configuration	`parallelODE=true`	`=false`	helps?
Low-order Picard (2D)	46.9	46.9	no
JFNK + `diffusion` (2D)	57.06	57.58	no
JFNK + `none` (2D)	28.76	28.96	no
`gaussPointODE` + HO p3 (2D, loop)	29.259	29.262	no
3D, 17 280 cells (loop)	2.861	2.816	no

The two scenarios where a benefit could be expected were specifically tested: `gaussPointODE` with high order p3 (several ODEs per cell, ∼16× more ionic work) and 3D with many cells. **Conclusion.** In every case the result is identical between `true`/`false` (and the CPU % does not change), i.e. the parallel region of the ODE contributes ∼ 0. The reason is that the ionic ODE (RKF45 on TNNP, above all at rest) is a *small fraction* of the cost: what dominates is the LRE reconstruction at the Gauss points, the averaging of I_{ion} and the high-order assembly/solve, which are serial operations that grow equally or more when the problem is enlarged. C1 is functionally correct (bit-identical results) but does not speed up this problem profile; it would only pay off with much stiffer models or extensive activation fronts.

F.4 Internal PETSc/BLAS threading (counterproductive in Picard)

Picard, `parallelODE`=`false`, varying `OMP_NUM_THREADS` (20 steps, corner).

`OMP_NUM_THREADS`	wall [s]	CPU	User [s]
1	45.7	98%	44.9
20	47.3	197%	68.8

Conclusion. With more threads, internal PETSc/BLAS/Eigen parallelism appears (∼ 2× CPU), but the wall time does not drop (it rises slightly): it is *memory-bound* and only wastes CPU. Picard should be run with `OMP_NUM_THREADS=1`.

F.5 Linear solver: AMG vs. ILU (the solve is not the bottleneck)

Comparison of `petscLinearKspType`/`petscLinearPcType` with `OMP_NUM_THREADS=1`.

Case	Linear solver	`loopWall` [s]	lin. iter. (total)
3D, 17 280, Δt=10^{-6}, low order	`gmres+ilu`	5.27	16
	`cg+gamg`	5.72	24
2D, 23 726, Δt=10^{-4}, HO consistent	`gmres+ilu`	70.21	40
	`gmres+gamg`	70.66	72
	`cg+gamg`	70.47	72

Conclusion. AMG is slightly slower and uses *more* iterations: with small Δt the system is mass-dominated ($\mathbf{A} \approx \mathbf{M}/\Delta t$), well conditioned and reaction-like, where ILU already is an excellent local preconditioner and the multigrid hierarchy is counterproductive. More importantly still: the `loopWall` is practically the same ($\sim$ 70 s) whatever the linear solver (40 vs. 72 iterations), which shows that **the linear solve is not the bottleneck**. Together with the previous experiments, the dominant cost in the high-order Picard regime is the high-order machinery itself (LRE reconstruction and Gauss-point handling), not the ODE nor the linear solver.

F.6 Synthesis and recommendations

- Keep `parallelODE`=`false` and `OMP_NUM_THREADS=1`: neither C1 nor the internal threading speeds up this problem.
- Keep `petscLinearPcType`=`ilu` (do not use AMG): the system is well conditioned and the linear solve is not the bottleneck.
- The real gains, without parallelism, are Phase B (factorisation reuse, $\sim$ 9% in Picard, identical results) and A3 (`jfnkPreconditioner`=`diffusion`, 1.67$\times$ in JFNK).
- The natural target to speed up the high-order regime would be to parallelise or make cheaper the LRE reconstruction and the Gauss-point handling (not done here).

References

[1] K. H. W. J. ten Tusscher, D. Noble, P. J. Noble, A. V. Panfilov, *A model for human ventricular tissue*, AJP – Heart and Circulatory Physiology **286** (2004) H1573–H1589.

[2] S. A. Niederer et al., *Verification of cardiac tissue electrophysiology simulators using an N-version benchmark*, Prog. Biophys. Mol. Biol. **104** (2012) 22–48.

[3] D. A. Knoll, D. E. Keyes, *Jacobian-Free Newton-Krylov methods: a survey of approaches and applications*, J. Comp. Phys. **193** (2004) 357–397.

[4] J. Nocedal, S. J. Wright, *Numerical Optimization*, 2nd edition, Springer, 2006.

[5] Y. Saad, *Iterative Methods for Sparse Linear Systems*, 2nd edition, SIAM, 2003.

[6] E. Hairer, G. Wanner, *Solving Ordinary Differential Equations II*, Springer, 1996.